

Weight Smoothing in Next-to-Leading-Order Monte Carlo Simulations using Gradient Boosted Decision Trees

Emin Yuksel

May 2026

Abstract

This project introduces an advanced machine learning framework using Gradient Boosted Decision Trees (XGBoost) to address the unphysical negative weight problem inherent in Next-to-Leading-Order (NLO) Monte Carlo event simulations. By implementing a custom asymmetric loss function and optimizing structural constraints, the pipeline successfully smooths out rapid weight fluctuations while strictly preserving global physical cross-sections for Drell-Yan and fully leptonic $t\bar{t}$ production channels.

Contents

1	Introduction	4
2	Theoretical Background and the Negative Weight Problem	5
2.1	Monte Carlo Event Generators	5
2.1.1	Direct Monte Carlo	5
2.1.2	Monte Carlo Integration	6

2.1.3	Unweighted Integration Behavior	7
2.1.4	Weighted Integration Behavior	7
2.2	Importance Sampling	8
2.3	Stratified Monte Carlo	8
2.4	The VEGAS Algorithm	9
2.5	QCD Perturbation Theory and NLO Calculations	12
2.5.1	The Perturbative vs. Non-Perturbative Regimes	12
2.6	Double Counting and the Matching Framework	13
2.6.1	The Double Counting Problem	13
2.6.2	The MC@NLO Matching Solution	14
2.7	The Origin of Negative Weights in MC@NLO	14
2.7.1	The Subtraction Framework and Negative Weights	15
2.7.2	Properties and Statistical Cost of Negative Weights	16
2.8	Physical Case Studies: Drell-Yan and $b\bar{b}$ Production	16
2.8.1	Drell-Yan Production	16
2.8.2	$b\bar{b}$ Production	17
3	Methodology and Machine Learning Framework	18
3.1	Traditional Mitigations: Born Spreading and Overlapping Elimination	18
3.1.1	Regression	19
3.1.2	Decision Trees	23
3.1.3	The Four-Step Decision Tree Optimization Pipeline	25
3.1.4	Logistic Regression vs Decision Tree	27
3.1.5	Boosted Decision Trees (BDT)	29
3.1.6	Comparison: Single Trees vs. BDTs	30

3.1.7	Hyperparameter Optimization: Tree Depth and Feature Stability	32
3.1.8	Deep Learning (Neural Networks)	33
3.1.9	Boosted Decision Trees vs. Deep Learning	35
3.2	Gradient Boosted Decision Trees (XGBoost)	36
3.3	Feature Engineering and Target Identification for $b\bar{b}$ and Drell-Yan	36
3.3.1	Features (Input X)	37
3.3.2	Target (Label y)	37
4	Results and Performance Evaluation	38
4.1	Dataset Generation and Monte Carlo Setup	38
4.2	Single-Process Smoothing Results	38
4.2.1	Drell-Yan Production (Electroweak Baseline)	38
4.2.2	$b\bar{b}$ Pair Production (Pure QCD)	43
4.2.3	$t\bar{t}$ Pair Production (Heavy Mass Decay Regime)	48
4.3	Multi-Process Evaluation and Combined Training	55
4.3.1	The Monolithic Training Attempt	55
4.3.2	The Modular Inference Solution	60
4.4	Computational Efficiency and Effective Sample Size (ESS)	64
5	Conclusion and Future Outlook	66

1 Introduction

High-precision predictions are fundamental to the success of the physics program at the Large Hadron Collider (LHC). As experimental measurements reach unprecedented levels of statistical precision, theoretical predictions must keep pace to enable accurate data interpretation and new physics searches. This demands the use of Next-to-Leading-Order (NLO) and Next-to-Next-to-Leading-Order (NNLO) perturbative QCD calculations, which are standard components of modern Monte Carlo (MC) event generators.

While NLO simulations drastically reduce theoretical scale uncertainties and provide a more accurate description of complex final states, they introduce a severe computational burden. To systematically cancel out infrared divergences, subtraction formalisms must be used. This process yields a fraction of simulated events that carry negative statistical weights. Unlike physical events, these negative weights do not correspond to actual physical processes but are mathematical artifacts of the perturbation theory framework.

The presence of negative weights severely degrades the statistical efficiency of the simulated samples. To achieve a targeted statistical precision, experimental collaborations must generate, simulate, and reconstruct significantly larger numbers of events, a process that consumes vast amounts of computing storage and CPU time. For high-luminosity processes like Drell-Yan or $b\bar{b}$ production, this inefficient resource allocation is rapidly becoming unsustainable.

To mitigate this problem, this project explores a machine learning approach to "smooth" these weight fluctuations. By training Gradient Boosted Decision Trees (specifically XGBoost) with a custom asymmetric loss function, we aim to reweight the phase space. This framework smooths out unphysical local fluctuations and eliminates negative weights while strictly preserving the global physical cross-sections and kinematic distributions essential for precision analysis.

2 Theoretical Background and the Negative Weight Problem

In high-energy physics, predicting the outcomes of particle collisions requires evaluating high-dimensional phase-space integrals dictated by quantum field theory. Because these integrals cannot be solved analytically due to the complexity of the matrix elements and final-state kinematics, numerical methods are essential. This section outlines the foundational concepts of Monte Carlo integration and its evolution into modern event generators.

2.1 Monte Carlo Event Generators

Broadly defined, any numerical technique that relies primarily on the formulation of pseudo-random numbers to solve deterministic or stochastic problems is classified as a Monte Carlo (MC) method. In the context of computational physics and event simulation, MC techniques can generally be categorized into three distinct frameworks based on their algorithmic complexity and objective:

- **Direct Monte Carlo:** Used for the explicit, step-by-step simulation of physical processes.
- **Monte Carlo Integration:** Used for the numerical evaluation of complex, multi-dimensional integrals over a defined phase space.
- **Importance Sampling Algorithms (e.g., VEGAS or Metropolis):** Adaptive techniques designed to optimize integration efficiency by concentrating sampling points in regions where the integrand is highly dynamic.

2.1.1 Direct Monte Carlo

Direct Monte Carlo techniques utilize random numbers to directly simulate the microscopic behavior of a complex system based on known probability distributions. Instead of solving macroscopic transport equations, the algorithm tracks individual trajectories. For analogy, in macroscopic systems like traffic flow, the acceleration, braking, and lane-changing behavior of individual vehicles can be governed by random numbers drawn from empirical distributions. In particle physics, this approach is mirrored when simulating the stochastic interactions of particles traversing a detector material, where processes such as Bremsstrahlung or ionization energy loss are sampled step by step.

2.1.2 Monte Carlo Integration

When the primary goal shifts from tracking individual particles to calculating total scattering cross-sections, Monte Carlo integration becomes the standard tool. This mathematical framework evaluates definite integrals by sampling the integrand at random points within the integration volume.

To illustrate, consider a one-dimensional function $f(x)$ bounded between the limits a and b . The standard formulation for estimating the integral I using N randomly generated points is given by:

$$I = \frac{b-a}{N} \sum_{i=1}^N w_i f(x_i)$$

where:

- x_i : A random number generated uniformly within the domain $[a, b]$.
- w_i : The statistical weight assigned to the event, which scales the contribution of each sampled point to the total sum (for uniform sampling, $w_i = 1$).

As $N \rightarrow \infty$, the central limit theorem guarantees that this sum converges to the true value of the integral, with a statistical uncertainty that scales as $1/\sqrt{N}$. In a realistic matrix element calculation, the integrand $f(x)$ is replaced by the differential cross-section $d\sigma/d\Phi$, and the integration domain spans the multi-dimensional phase space of the final-state particles.

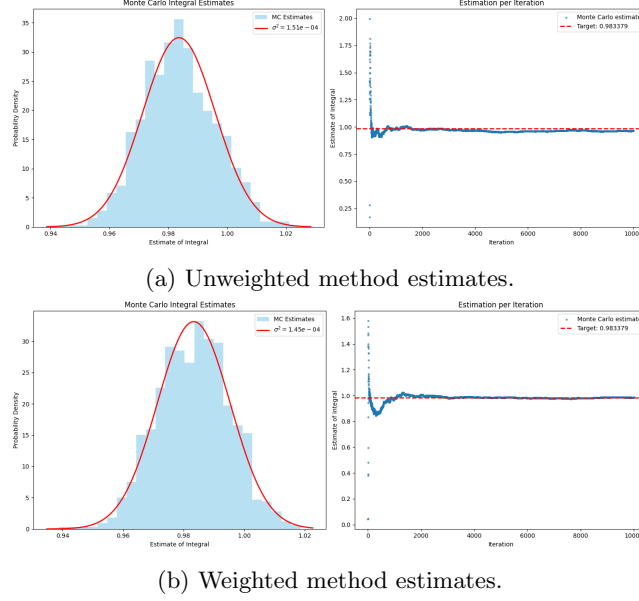


Figure 1: Comparison of numerical convergence and stability between the Unweighted and Weighted Monte Carlo integration methods.

To understand how event weights affect the precision of a simulation, we ran a simple numerical experiment. We compared a standard unweighted Monte Carlo integration method against a weighted integration method. Figure 1 shows how these two methods converge and how stable their final results are.

2.1.3 Unweighted Integration Behavior

As shown in Figure 1a, the unweighted method treats every sampled region with the exact same geometric importance. The scatter plot on the left shows a standard $1/N$ convergence path where the estimates gradually level off toward the true target value (the red dashed line). Repeating this simulation multiple times creates a standard Gaussian distribution of final estimates (the histogram on the right) with a baseline statistical variance (σ^2).

2.1.4 Weighted Integration Behavior

In contrast, Figure 1b introduces a weighted integration approach by applying a weight factor ($[e^{-x_n}]$) that closely matches the shape of the integrated function. This completely changes how the simulation behaves. As seen in the left scatter plot, the estimate curve changes rapidly in early iterations because the

simulation places much higher statistical importance on the specific events that matter most.

Because the simulation prioritizes these high-importance regions, the calculation stabilizes much more efficiently than the unweighted approach. As a result, repeating the weighted simulation multiple times yields a much narrower and sharper Gaussian distribution (the histogram on the right). This directly proves that using a properly weighted system successfully lowers the final statistical variance (σ^2) and increases the overall precision of the integration.

2.2 Importance Sampling

Instead of selecting points from a completely uniform random distribution, the Importance Sampling technique alters the Probability Density Function (PDF) to mirror the shape of the function being integrated. Ideally, we want the chosen sampling distribution, $g(x)$, to be as close to the target function, $f(x)$, as possible:

$$\frac{f(x)}{g(x)} \sim 1 \tag{1}$$

The mathematical formulation for the Importance Sampling integral is expressed as:

$$\int_a^b f(x) dx = \int_a^b g(x) \left[\frac{f(x)}{g(x)} \right] dx \tag{2}$$

By restructuring the integral this way, the simulation generates significantly more sample points in the specific regions where the function contributes the most to the total value. Because the points are clustered where they matter most, this method drastically reduces the final statistical variance and improves the overall efficiency of the calculation.

2.3 Stratified Monte Carlo

Stratified Monte Carlo integration is another powerful technique designed to reduce statistical error. Instead of calculating the entire integral across the full volume all at once, the total integration space is split into multiple smaller sub-volumes, or "strata." The integral within each sub-volume is calculated independently, and the separate results are merged at the end to find the final total value.

This method is highly effective for functions with sharp spikes or sudden changes. If a significant portion of the integral's value comes from a very small region, a standard uniform simulation might only drop a few random points there by chance, leading to a massive statistical error. Stratified sampling solves this problem by forcing the simulation to dedicate a set number of points to every sub-volume, ensuring that high-contribution regions are never missed or under-sampled.

2.4 The VEGAS Algorithm

The VEGAS algorithm is one of the most widely used frameworks for multi-dimensional Monte Carlo integration and event generation. It combines the principles of importance sampling and stratified sampling to automatically optimize the simulation grid. In tools like MadGraph5_aMC@NLO, VEGAS operates in three main sequential steps:

1. **Grid Setup (Adaptive Optimization):** The algorithm begins by dividing the integration phase space into a uniform starting grid. It randomly samples a batch of points across this space to evaluate the behavior of the target function. Based on where the function shows the largest contributions or fluctuations, VEGAS dynamically adjusts the grid increments, making the grid cells smaller and denser in high-importance regions. Over successive iterations, this optimized grid forms a well-defined, normalized Probability Density Function, $g(x)$.
2. **Determining the Unweighting Bound:** Once the adaptive grid is stable, the algorithm calculates a maximum boundary value, h , across the phase space. This boundary value is defined by the maximum ratio of the target physics function, $f(x)$, to the optimized grid distribution, $g(x)$:

$$h = \max \left[\frac{|f(x)|}{g(x)} \right] \quad (3)$$

This upper bound establishes the roof needed to safely execute the subsequent unweighting step.

3. **Unweighting via Rejection Sampling:** In raw mathematical integration, every generated event carries a continuous, fluctuating weight. However, physical detectors and real-world particle interactions are inherently unweighted (meaning an event either physically occurs or it does not). To make the simulation realistic, VEGAS uses rejection sampling to convert these continuous weights into a binary format:

- A random number, R , is selected uniformly between 0 and h .
- The algorithm compares R against the event's weight ratio, $|f(x)|/g(x)$.

- If $R < |f(x)|/g(x)$, the event is accepted and assigned a standard weight of 1.
- If $R > |f(x)|/g(x)$, the event is rejected and completely discarded from the final dataset.

Finally, because certain Next-to-Leading-Order (NLO) quantum corrections introduce negative interference, a plus or minus sign ($\pm$) is reapplied to the accepted unit weights based on the original sign of $f(x)$. Due to the high number of discarded points needed to find events that pass the threshold, this unweighting stage stands as the most computationally demanding part of the generation pipeline.

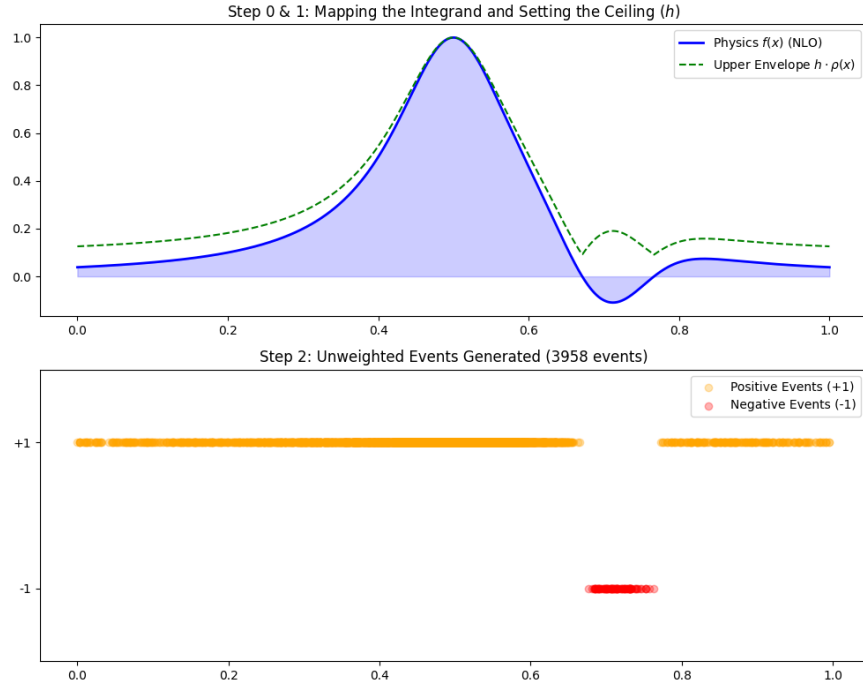


Figure 2: Numerical Python simulation illustrating the sequential operational phases of the VEGAS adaptive unweighting algorithm for an analytical Next-to-Leading-Order (NLO) toy model function. The top panel shows the target physics function $f(x)$ featuring a positive resonance peak and a deep negative subtraction pocket alongside the dynamically optimized adaptive upper envelope ($h \cdot \rho(x)$). The bottom panel displays the final resulting unweighted events ($w = \pm 1$) produced via rejection sampling, mapped to their respective discrete kinematic phase-space coordinates.

To visually demonstrate the step-by-step inner workings of the VEGAS algorithm, a Python numerical simulation was developed using an analytical toy

model. This model mimics a typical Next-to-Leading-Order (NLO) physics calculation, featuring a sharp positive resonance peak and a deep negative subtraction region. Figure 2 shows the clear progression from the continuous mathematical function to discrete, unweighted particle events.

The top panel of Figure 2 shows the first phase of the algorithm (Steps 0 and 1). The solid blue curve represents the continuous physics function, $f(x)$, which contains both positive values and a deep valley of negative corrections. The dashed green line shows the optimized upper envelope, calculated by multiplying the adaptive grid probability density, $\rho(x)$, by the maximum ratio bound, h .

As shown in the plot, the adaptive grid works perfectly: it automatically changes its shape to match the absolute magnitude of the physics function. The grid becomes tall and narrow over the positive peak and dips down to closely follow the shape of the negative pocket. This adaptive tailoring ensures that the ceiling sits safely above the function across the entire phase space while minimizing empty, wasted space.

The bottom panel displays the final result of the unweighting phase (Step 2) after running 5,000 sampling trials. Through the acceptance-rejection test, the continuous, fluctuating function is successfully converted into a discrete set of realistic event outputs.

Because the algorithm tracks the absolute value of the function, events are heavily generated both under the positive peak and inside the negative valley. The final output is completely binary: events that land in the positive region are assigned a clean unit weight of $+1$ (orange scatter points), while events that land in the subtraction pocket are assigned a unit weight of -1 (red scatter points). This plot perfectly illustrates the physical starting point of our project: the unweighted events in the negative band are the exact unphysical negative weights that our Gradient Boosted Decision Tree pipeline aims to smooth out.

2.5 QCD Perturbation Theory and NLO Calculations

To simulate realistic particle collisions, event generators must bridge the gap between pure quantum mechanical theory and the complex hadronic final states observed in real-world detectors. This simulation pipeline is generally broken down into three distinct phases:

1. **The Hard Scatter (Perturbative Phase):** The process begins when two high-energy partons (quarks or gluons inside the colliding protons) undergo a hard scattering interaction, typically at energy scales well above 100 GeV. This localized, high-energy core interaction can be calculated exactly using Feynman diagrams and quantum field theory matrix elements. The output of this stage consists solely of fundamental partons and leptons.
2. **The Parton Shower (Evolution Phase):** As the high-energy partons fly away from the initial interaction point, they accelerate and radiatively emit gluons, which in turn can split into quark-antiquark pairs. Simulation frameworks like `Pythia` calculate the probabilities of these sequential branchings. As the shower evolves, the total system energy is divided among an increasing number of partons. This branching process continues until the momentum scale of the partons drops down to a specific "cutoff scale" (typically around 1 GeV).
3. **Hadronization (Non-Perturbative Phase):** Due to the laws of color confinement in Quantum Chromodynamics (QCD), isolated quarks and gluons cannot exist freely in nature; real-world detectors only observe color-neutral hadrons. Once the parton shower drops below the 1 GeV threshold, phenomenological frameworks such as the Lund String Model take over. These models act as a bridge, mathematically binding the final partons together into physical hadrons to accurately mimic a real detector signal.

2.5.1 The Perturbative vs. Non-Perturbative Regimes

When a matrix element generator like MadGraph calculates a physical cross-section (σ), it sets up a perturbative series expansion based on powers of the strong coupling constant, α_s :

$$\sigma = c_0(\alpha_s^0) + c_1(\alpha_s^1) + c_2(\alpha_s^2) + c_3(\alpha_s^3) + \dots \quad (4)$$

The mathematical behavior of this series depends entirely on the energy scale of the interaction, splitting QCD analysis into two distinct regimes:

- **The Perturbative Region (High Energy):** At high energy scales, the strong coupling constant is very small ($\alpha_s \ll 1$). Because the baseline value is small, higher powers of α_s (such as α_s^2 and α_s^3) shrink rapidly, allowing the series to converge efficiently. Under these conditions, the Leading Order (LO) and Next-to-Leading-Order (NLO) terms dominate the calculation, making it mathematically safe to truncate the infinite series after the first few terms.
- **The Non-Perturbative Region (Low Energy):** As the energy scale drops below approximately 1 GeV, the strong coupling constant grows significantly ($\alpha_s > 1$). In this low-energy regime, raising α_s to higher powers causes the terms to grow rather than shrink, making the mathematical series diverge completely. Because higher-order terms become infinitely larger, standard perturbative calculations break down entirely. This divergence is why phenomenological models must replace exact matrix elements during the low-energy hadronization stage.

2.6 Double Counting and the Matching Framework

When simulating high-energy particle collisions, event generators combine two different theoretical tools to construct the full final state: Matrix Elements (ME) and Parton Showers (PS). While both tools calculate how partons radiate, they operate under completely different mathematical frameworks:

- **Matrix Elements (ME):** Calculated by tools like `MadGraph5_aMC@NLO`, this approach uses exact quantum field theory and Feynman diagrams. It provides a highly accurate description of hard, high-energy particle emissions. However, as established previously, it fails at low energies where the strong coupling constant (α_s) becomes large.
- **Parton Showers (PS):** Handled by programs like `Pythia` or `Herwig++`, this approach uses a probabilistic method to simulate successive, lower-energy particle radiations. It relies on a combination of theoretical splitting functions and tuning parameters from past experimental data, allowing it to remain stable and reliable even when α_s grows large.

2.6.1 The Double Counting Problem

The overlap between ME and PS creates a serious challenge known as the double counting problem. This issue arises during the generation of Next-to-Leading-Order (NLO) corrections.

As shown in Figure 3, an NLO calculation consists of a Leading-Order Born level core (a), virtual internal loop corrections (b), and real emission processes

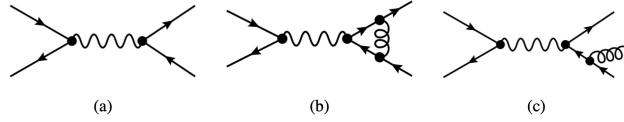


Figure 3: LO and NLO production processes: (a) Leading-order (LO) Born level, (b) NLO with virtual corrections, and (c) NLO with real emission.

where an extra hard parton is radiated into the final state (c). The problem occurs because the Matrix Element explicitly calculates this extra real emission parton as a hard, well-separated particle. However, when the event is passed to the Parton Shower program, the shower’s probabilistic algorithms can independently generate an almost identical soft or collinear parton emission from the core process.

Because the boundary between the Matrix Element and the Parton Shower allows both systems to simulate the exact same physical radiation, certain types of events are accidentally produced twice. This unphysical overlap severely distorts the final simulated cross-section and distributions.

2.6.2 The MC@NLO Matching Solution

To overcome this issue, event generators use a formal matching procedure. In the MC@NLO framework, this is achieved by performing a precise mathematical subtraction before the simulation begins.

The algorithm first isolates the exact probability distribution of what the Parton Shower program is expected to radiate on its own. Then, the framework explicitly subtracts this shower emission distribution directly from the Matrix Element’s real-emission calculation. By removing the expected shower contributions from the hard matrix elements ahead of time, the framework completely eliminates the unphysical overlap. This ensures that the transition from the high-energy Matrix Element region to the low-energy Parton Shower region is completely seamless and mathematically correct.

2.7 The Origin of Negative Weights in MC@NLO

Negative weights are not a physical phenomenon; instead, they are mathematical byproducts of how perturbation theory is calculated on a computer. To evaluate a cross-section at Next-to-Leading-Order (NLO), we must sum two distinct types of quantum corrections across the available phase space:

$$\sigma_{\text{NLO}} = \int d\phi_{n+1} |\mathcal{M}_{\text{real}}|^2 + \int d\phi_n |\mathcal{M}_{\text{virtual}}|^2 \quad (5)$$

where $|\mathcal{M}_{\text{real}}|^2$ represents the matrix element for a real parton emission (an $n+1$ body final state), and $|\mathcal{M}_{\text{virtual}}|^2$ represents the internal virtual loop corrections (an n body final state).

Individually, both of these integrals contain severe mathematical infinities (divergences). For the real emission term, an infinity occurs whenever a radiated gluon is emitted with zero energy (a soft divergence) or at a zero-degree angle relative to the parent parton (a collinear divergence). The virtual term contains matching internal loop divergences that approach negative infinity.

According to the Kinoshita-Lee-Nauenberg (KLN) theorem, if these two terms are integrated over the entire phase space together, these positive and negative infinities perfectly cancel each other out, leaving behind a finite, physically meaningful cross-section:

$$\sigma_{\text{physical}} = \underbrace{\sigma_{\text{virtual}}}_{-\infty} + \underbrace{\sigma_{\text{real}}}_{+\infty} = \text{Finite Value} \quad (6)$$

2.7.1 The Subtraction Framework and Negative Weights

While the final sum is mathematically finite, computers cannot handle or calculate two individual infinite terms. The values must be made finite before numerical integration can begin. To achieve this, event generators introduce an artificial counter-term, $|\mathcal{M}_{\text{sub}}|^2$. This counter-term is designed to mimic the exact singular behavior of the real emission term, but it is simple enough to be integrated analytically.

By adding and subtracting this counter-term, the NLO cross-section formula is rearranged into two completely finite pieces that a computer can safely handle:

$$\sigma_{\text{NLO}} = \underbrace{\int d\phi_{n+1} (|\mathcal{M}_{\text{real}}|^2 - |\mathcal{M}_{\text{sub}}|^2)}_{\text{Finite Real Term}} + \underbrace{\int d\phi_n |\mathcal{M}_{\text{virtual}}|^2 + \int d\phi_{n+1} |\mathcal{M}_{\text{sub}}|^2}_{\text{Finite Virtual Term}} \quad (7)$$

Across most regions of the physical phase space, the real emission probability is larger than the counter-term ($|\mathcal{M}_{\text{real}}|^2 > |\mathcal{M}_{\text{sub}}|^2$), resulting in standard positive weights. However, in specific soft and collinear regions, the counter-term can overcompensate for the real emission matrix element ($|\mathcal{M}_{\text{real}}|^2 < |\mathcal{M}_{\text{sub}}|^2$). When this over-subtraction occurs, the computer records a negative-weight event.

2.7.2 Properties and Statistical Cost of Negative Weights

The presence of negative weights introduces several severe computational and statistical challenges:

- **Statistical Cancellation:** Negative-weight events act as a destructive interference in histograms. Instead of adding to a data bin, they subtract from it, reducing the visible event count and increasing the statistical uncertainty of the simulation.
- **Sample Size Degradation:** If a simulation contains a fraction, r , of negative-weight events, the total number of events required to reach the same statistical precision as an unweighted sample scales by a factor of:

$$\text{Simulation Cost Factor} = \frac{1}{(1 - 2r)^2} \quad (8)$$

For example, a simulation with a 25% negative weight fraction ($r = 0.25$) requires generating 4 times as many total events to achieve the same statistical accuracy as a purely positive sample.

- **Generator Specifics:** While negative events can be avoided in certain processes using alternative matching frameworks like POWHEG, that method is only available for a limited subset of simple physics channels.
- **Presence at Leading-Order:** Although primarily an NLO issue, negative weights can occasionally appear in Leading-Order (LO) simulations due to quantum interference terms or non-positive-definite parton density functions (PDFs).

2.8 Physical Case Studies: Drell-Yan and $b\bar{b}$ Production

To test and validate our weight-smoothing framework under different physical conditions, we choose two distinct processes as benchmarks: Drell-Yan production and bottom-quark pair ($b\bar{b}$) production. These channels represent two very different regimes of quantum interactions and final-state radiation patterns.

2.8.1 Drell-Yan Production

The Drell-Yan process occurs when two high-energy quarks from the colliding protons annihilate to produce a virtual photon or a Z boson, which then immediately decays into a pair of leptons (l^+l^-), as illustrated in Figure 4a.

While the initial state of this interaction involves strong interactions (QCD), the core process and the final state are governed by electroweak physics. Because the final-state leptons do not carry color charge, they do not emit gluons or initiate a parton shower. This clean final state prevents any extra QCD radiation from contaminating the primary hard process. As a result, the negative-weight problem in Drell-Yan is relatively mild and concentrated in well-defined kinematic regions.

2.8.2 $b\bar{b}$ Production

In contrast, $b\bar{b}$ production ($pp \rightarrow b\bar{b}$) is a pure QCD process dominated heavily by gluon-gluon fusion, as shown in Figure 4b.

Because this interaction is driven entirely by the strong force, the final state contains two massive bottom quarks that carry color charge. As these b -quarks fly apart, they can easily radiate additional gluons, initiating their own secondary parton showers. This means the gluon emissions from the hard scatter become heavily mixed and contaminated with the radiation from the $b\bar{b}$ shower. Due to this intense color-connected radiation and the relatively low mass of the b -quark, the NLO subtraction terms experience severe cancellations, generating a significantly more complex and severe negative-weight problem.

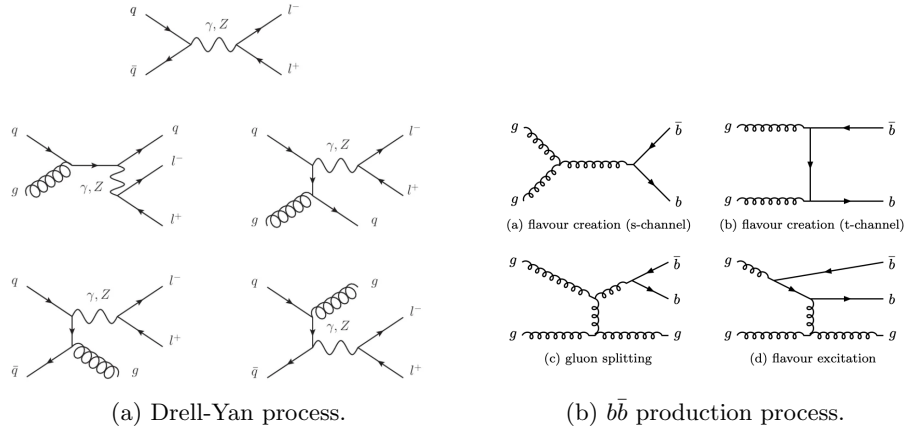


Figure 4: Feynman diagrams representing the lowest-order configurations for our two physical case studies: (a) the electroweak Drell-Yan lepton-pair production and (b) the strong interaction $b\bar{b}$ pair production.

3 Methodology and Machine Learning Framework

3.1 Traditional Mitigations: Born Spreading and Overlapping Elimination

Before developing machine learning solutions, traditional methods were created to handle the negative-weight problem within event generators. The most common of these baseline techniques is Born Spreading.

In a standard Next-to-Leading-Order (NLO) calculation, the counter-term is subtracted from the exact same mathematical point in the phase space where the real emission divergence occurs. Because these subtractions are extremely sharp and highly localized, they leave behind deep valleys of large negative weights next to massive positive spikes.

Born Spreading solves this issue by mechanically smearing out the counter-term subtraction across a wider area of the phase space. Figure 5 uses a numerical simulation to demonstrate exactly how this smearing process changes the event weight distribution and improves simulation performance.

Smearing the Subtraction (Top Panel): The top panel of Figure 5 displays the effect of Born Spreading on the raw event weights. The solid red line shows the raw NLO calculation, where a sharp, localized subtraction creates a deep spike that plunges far below zero into the unphysical negative-weight region.

The solid green line shows the event weight distribution after Born Spreading is applied. By widening the phase-space area of the counter-term, the subtraction is distributed smoothly. As a result, the large positive spikes and deep negative valleys cancel each out locally. This smoothing drastically reduces the overall fraction of negative weights, which directly increases the Effective Sample Size (ESS) of the generated dataset.

Impact on Unweighting Efficiency (Bottom Panel): The bottom panel illustrates the secondary benefit of Born Spreading: its direct impact on the unweighting efficiency during event generation. As established previously in the VEGAS algorithm discussion, the speed of unweighting depends entirely on the maximum absolute weight height (h), which forms the ceiling for the acceptance-rejection test.

The raw, un-smeared calculation produces a massive peak value (h_{raw}), shown by the dotted red line. This requires the generator to build a very tall sampling box, meaning the vast majority of generated points will fall below the line and be rejected. By spreading out the subtraction, the green curve lowers the maximum

peak value to a much shorter ceiling (h_{spread}). Lowering this maximum height reduces the amount of empty space in the sampling box, significantly increasing the number of accepted events and accelerating the simulation runtime.

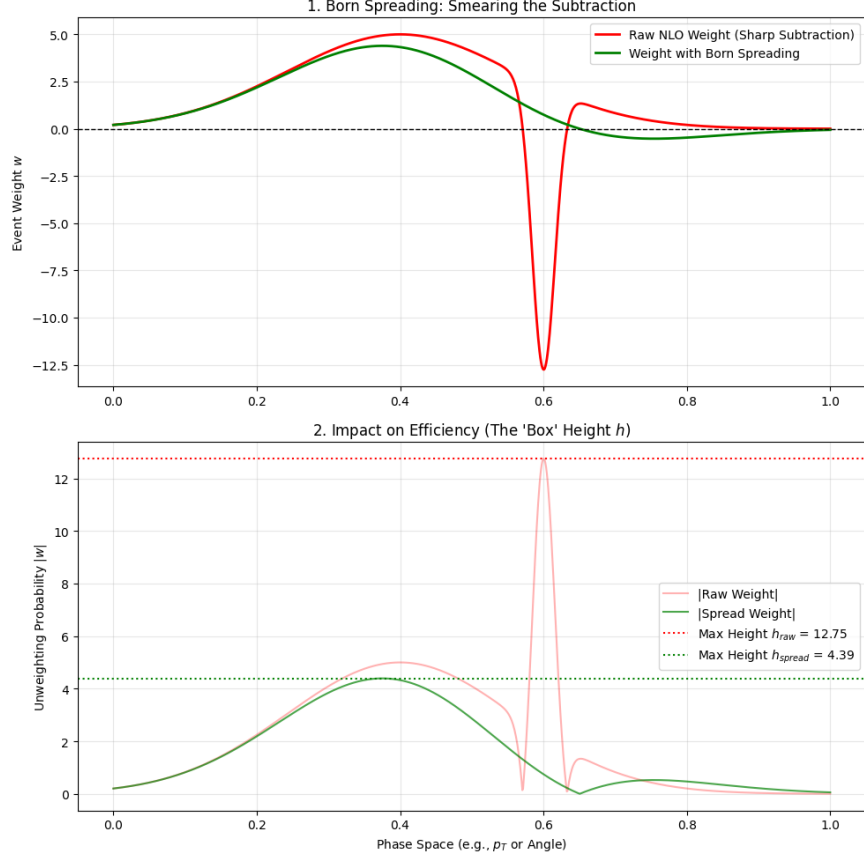


Figure 5: Numerical simulation demonstrating the mechanics of the traditional Born Spreading framework. The top panel shows how smearing out a sharp, localized NLO subtraction across a wider phase space eliminates deep negative event weights. The bottom panel illustrates how this flattening process lowers the absolute maximum height ceiling (h), leading to a direct increase in unweighting efficiency.

3.1.1 Regression

Regression is a fundamental machine learning task focused on estimating the mathematical relationship between a dependent variable (the target) and one or more independent variables (the features). While classification models predict

discrete labels, regression models output continuous values, making them the appropriate tool for learning the continuous value of a physical cross-section or event weight.

Linear Regression

Linear regression is the simplest parametric approach to regression analysis. It assumes that the continuous target output, y , can be modeled as a direct linear combination of the input features, $\mathbf{x}$, scaled by a set of trainable model coefficients, β :

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n \quad (9)$$

where β_0 is the intercept parameter and $\hat{y}$ is the value predicted by the model.

To optimize the model and find the best fitting parameters (β), we must define a loss function to measure the error between the model predictions and the real data points. The most common loss function for standard regression tasks is the Mean Squared Error (MSE):

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (10)$$

The MSE objective works by squaring the individual residual errors, which ensures that larger deviations away from the true values are penalized much more heavily than minor fluctuations.

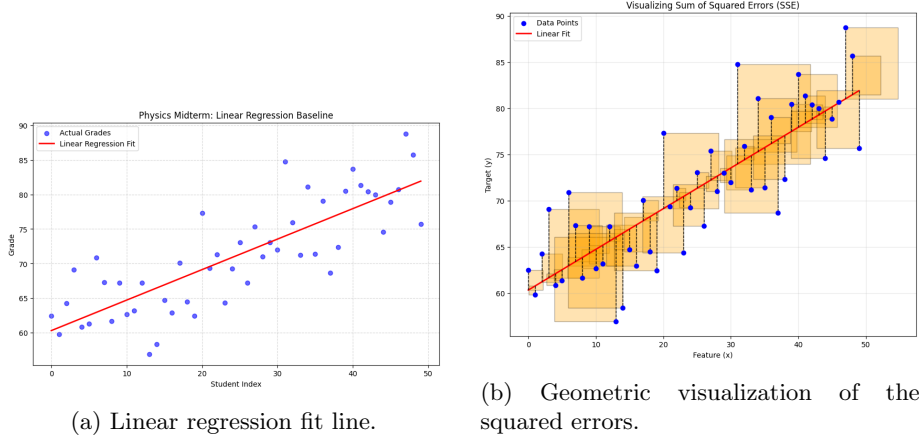


Figure 6: Baseline linear regression modeling on a simulated dataset. Panel (a) illustrates the optimized best-fit line tracking the overall upward trend of the data points. Panel (b) provides a geometric visualization of the Mean Squared Error (MSE) minimization process, where the area of each orange square represents the squared residual value $((y_i - \hat{y}_i)^2)$ for an individual point.

Figure 6 displays a numerical simulation demonstrating how a baseline linear regression model fits a dataset containing localized random noise.

As seen in Figure 6a, the optimized fit line easily captures the macroscopic linear trend of the points across the phase space. However, because a linear model is rigid, it cannot adapt to local shapes and simply cuts straight through the data.

Figure 6b illustrates the geometric meaning under the hood of this optimization. The vertical dashed lines track the raw residual error for each point, while the orange boxes represent the squared value of that error. The linear regression algorithm searches for the exact slope and intercept that minimizes the total combined area of all these orange boxes. This geometric baseline establishes a helpful contrast to the highly flexible, tree-based step functions we utilize later to map non-linear physical distributions.

Logistic Regression

While linear regression is designed to predict continuous values, logistic regression adapts this baseline framework to handle classification tasks. Instead of mapping features directly to an unbounded output, logistic regression calculates the probability that a given event belongs to a specific category (such as distinguishing a physics signal from a background process).

To ensure that the model output remains bounded between 0 and 1 (representing a valid mathematical probability), the linear combination of input features is passed through a non-linear activation function called the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = \beta^T x \quad (11)$$

Here, $\beta^T x$ represents the standard linear combination of your features and trainable parameters ($\beta_0 + \beta_1 x_1 + \dots$).

Because the target outputs in classification are discrete binary labels ($y_i \in \{0, 1\}$), Mean Squared Error is no longer a suitable objective function. Instead, logistic regression models are trained by minimizing the Binary Cross-Entropy (BCE) loss, also known as Log-Loss:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (12)$$

The Log-Loss function works by penalizing incorrect predictions asymptotically; the mathematical loss spikes severely if the model is highly confident about a wrong prediction.

Figure 7 displays the performance of this classification model using simulated data.

As shown in Figure 7a, the continuous S-shaped sigmoid curve smooths the transition between discrete data classes. By choosing a probability threshold of ≥ 0.5 , the model establishes a precise decision boundary that marks the exact

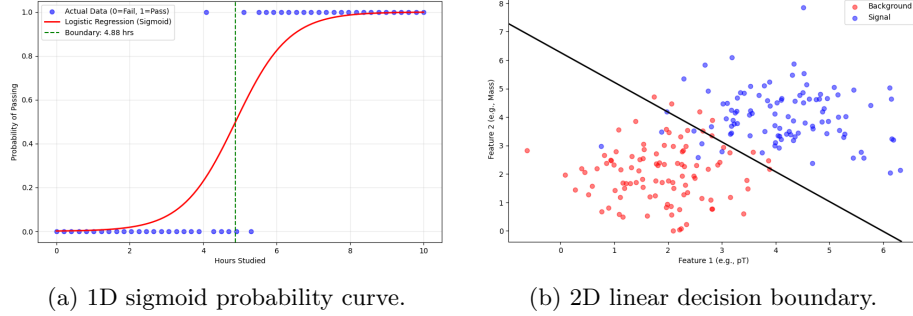


Figure 7: Numerical simulation of a logistic regression classifier. Panel (a) shows how the continuous sigmoid function maps a single feature to a pass-or-fail probability, setting a vertical decision boundary where the probability equals 0.5. Panel (b) illustrates how this threshold creates a rigid, flat linear decision boundary when separating two distinct event classes across a two-dimensional kinematic phase space.

point where a data sample transitions from a predicted failure to a predicted success.

When expanding this model to a two-dimensional phase space containing a signal and a background distribution, as shown in Figure 7b, this 0.5 threshold forms a flat, straight line. Because the mathematical argument inside the sigmoid function ($z = \beta^T x$) is strictly linear, the resulting decision boundary is forced to be a rigid linear separator. While this works well for cleanly separated datasets, it highlights the same limitation as linear regression: a rigid linear model cannot bend or warp to isolate complex, non-linear kinematic shapes, establishing the scientific need for the flexible decision tree ensembles used in this analysis.

Summary of Parametric Models

To conclude our analysis of classical parametric algorithms, it is helpful to directly contrast the primary differences between linear and logistic regression. Although both frameworks share a similar foundation rooted in linear combinations of input features, they serve entirely different analytical purposes. Linear regression functions as a tool for tracking continuous geometric trends, while logistic regression transforms those same underlying values to map non-linear boundaries for classification. Table 1 summarizes these key differences across their outputs, core equations, optimization loss functions, and spatial boundaries.

Table 1: Comparison between baseline Linear and Logistic Regression models.

Feature	Linear Regression	Logistic Regression
Output	Continuous (e.g., Event Weight)	Discrete / Probabilistic (e.g., Signal vs. Background)
Equation	$y = \beta^T x$	$y = \sigma(\beta^T x)$
Loss Function	Mean Squared Error (MSE)	Log-Loss (Binary Cross-Entropy)
Boundary	Continuous Trend Line	Rigid Separation Plane

3.1.2 Decision Trees

While logistic regression is constrained by a single rigid, linear boundary, Decision Trees provide a highly flexible non-parametric alternative for classification and regression tasks. Instead of using continuous mathematical weights or bias parameters to draw a separation plane, a decision tree operates as an automated "cuts" generator. It relies entirely on pure hierarchical logic to partition a complex feature space into localized, uniform regions.

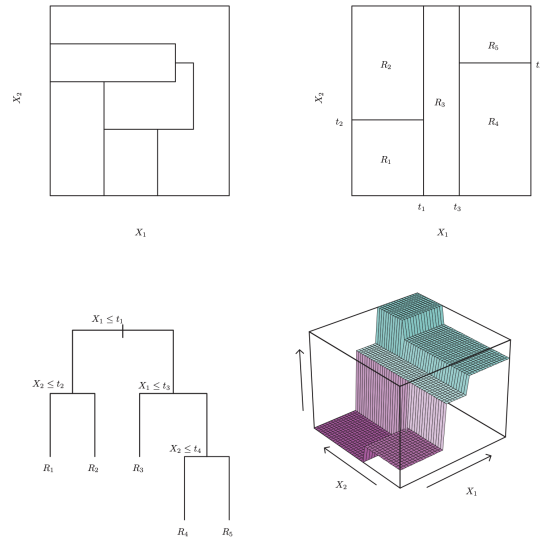


Figure 8: Geometric and architectural mechanics of a decision tree partition scheme across a two-dimensional phase space. The top panels illustrate the sequential orthogonal cuts (t_1 through t_4) dividing the space into distinct regions (R_1 through R_5). The bottom-left panel shows the corresponding hierarchical binary tree structure, and the bottom-right panel displays the resulting multi-dimensional step-function prediction surface.

Figure 8 provides a complete geometric visualization of how a decision tree builds its prediction boundaries across a two-dimensional phase space defined by features X_1 and X_2 .

3.1.3 The Four-Step Decision Tree Optimization Pipeline

Building an optimal decision tree requires balancing a model’s ability to learn real physical relationships against its tendency to memorize random noise. This optimization is carried out in four distinct steps:

Step 1: Tree Growth

The algorithm begins at the top root node containing the entire training dataset. Using a top-down, "greedy" approach known as recursive binary splitting, it evaluates every input feature X_j and every possible split threshold s . For any chosen pair (j, s) , the algorithm divides the data into two temporary regions:

$$R_1(j, s) = \{X \mid X_j < s\} \quad \text{and} \quad R_2(j, s) = \{X \mid X_j \geq s\} \quad (13)$$

The algorithm calculates the Residual Sum of Squares (RSS) for all possible configurations and selects the exact split that minimizes the total localized error:

$$\text{RSS} = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2 \quad (14)$$

where $\hat{y}_{R_m}$ is simply the mean value of the target points within that specific region. This splitting process repeats recursively for each new sub-node, stopping only when the final nodes drop below a predefined minimum sample size. This results in a massive, unconstrained tree.

Step 2: Cost-Complexity Pruning

Because the fully grown tree branches out until almost every training point is isolated, it inherently overfits the dataset by capturing statistical noise. To simplify the model, we perform cost-complexity pruning. We introduce a tuning parameter, α , to penalize over-complicated models by defining a penalized cost function:

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (15)$$

where $|T|$ is the total number of terminal leaf nodes in a given subtree, and α acts as the complexity penalty.

If $\alpha = 0$, no penalty is applied, leaving the original overgrown tree intact. As α increases, the penalty for having a large number of leaves grows, forcing the algorithm to systematically prune away weak, high-variance branches. By scanning across a range of α values, the algorithm generates a sequence of distinct, increasingly simpler subtrees.

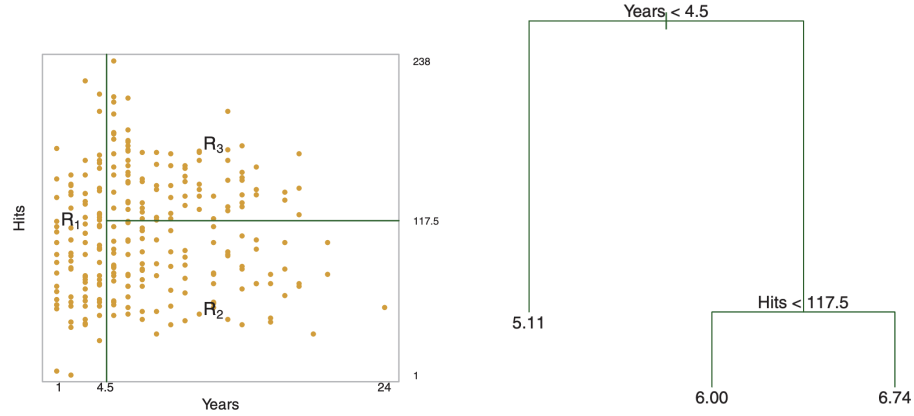
Step 3: Cross-Validation Optimization

To identify which pruned subtree from Step 2 will perform best on independent data, K -fold cross-validation is deployed. The training data is split into K equal blocks (folds). The model rotates through the dataset, training on $K - 1$ blocks and evaluating its validation Mean Squared Error (MSE) on the remaining fold.

This process is repeated across the entire sequence of subtrees generated by our different α candidates. The specific value of α that yields the lowest average validation MSE across all iterations is chosen as the winner. This optimal value represents the perfect structural balance: a tree complex enough to capture the true physical signal but simple enough to ignore background fluctuations.

Step 4: Subtree Extraction

In the final step, the winning α value selected during cross-validation is mapped back onto the main training pipeline. The algorithm extracts the exact corresponding subtree from Step 2, providing our final optimized model.



(a) Two-dimensional phase space splits. (b) Corresponding pruned tree structure.

Figure 9: A visualization of a fully optimized regression tree after cost-complexity pruning and cross-validation selection. Panel (a) shows the final stable spatial divisions (R_1 , R_2 , R_3) across a two-variable feature space. Panel (b) traces the clean, simplified binary decision path used to navigate an event to its final predicted numerical baseline value at the terminal leaf nodes.

Figure 9 shows an example of a final regression tree after completing this four-step optimization process.

As seen in the phase-space scatter plot (Figure 9a), the unconstrained, complex splits from the growth phase have been successfully pruned down to just two highly stable, orthogonal cuts. The first cut separates the data vertically at $\text{Years} = 4.5$, and the second splits the remaining events horizontally at $\text{Hits} = 117.5$, creating three clean prediction zones (R_1 , R_2 , and R_3).

Figure 9b displays the clean, easily readable logical tree structure that corresponds directly to these spatial boundaries. Any event entering the system is evaluated sequentially at the decision nodes, effortlessly routing it to one of the three final leaves (5.11, 6.00, or 6.74) representing the localized average value of that region. This example demonstrates how our pipeline strips away complex, overfitted branches to deliver a highly robust and interpretable model structure.

3.1.4 Logistic Regression vs Decision Tree

To select the ideal machine learning framework for phase-space weight smoothing, it is vital to contrast how logistic regression and decision trees build their internal decision boundaries. Table 2 provides a conceptual overview of their fundamentally different mathematical approaches.

Table 2: Conceptual comparison between Logistic Regression and Decision Tree models.

Model Class	Approach & Boundary Characteristics
Logistic Regression (Global)	Tries to find one single diagonal line that works for the whole dataset.
Decision Tree (Local)	Asks a series of “If-Then” questions to zoom in on the signal. It looks at a small region of the data, solves it, and moves to the next.

While logistic regression optimizes globally to draw a single smooth separator, decision trees look at local clusters of data, solving complex sub-regions one piece at a time. This difference leads to completely distinct visual behaviors depending on the true underlying distribution of the physics data.

Figure 10 illustrates this structural comparison across two separate data scenarios, mapping out exactly when each model excels or struggles.

Scenario 1: True Linear Boundary (Top Row 10)

The top-left panel shows a situation where the true physical boundary separating the two classes is perfectly linear and diagonal. A global model like logistic regression handles this flawlessly, mapping the exact sloped plane with a single clean cut.

In contrast, the top-right panel shows how a decision tree struggles in this environment. Because tree splits must be strictly perpendicular to a feature axis (orthogonal), it cannot make a diagonal line. Instead, it is forced to approximate the slope by building a jagged, stair-step sequence of multiple local cuts. While

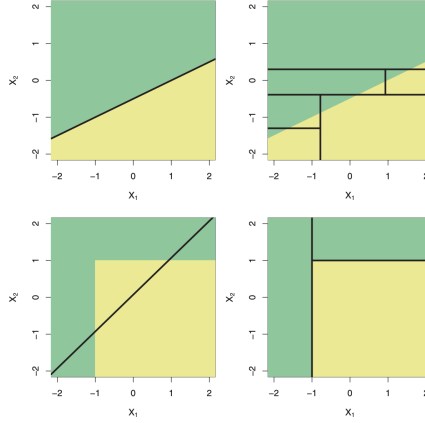


Figure 10: Visual decision boundary comparison between a linear model and a localized decision tree partition.

it can mimic the line if grown deeply, it requires significantly more parameters and becomes structurally inefficient.

Scenario 2: True Non-Linear/Box Boundary (Bottom Row 10)

The bottom-right panel displays the opposite scenario, where the true physical boundary is rectangular or box-shaped—a common occurrence in physics analysis when applying specific kinematic “window” selections (such as mass windows or momentum cuts). A decision tree isolates this region perfectly using just two clean, axis-aligned cuts.

Conversely, the bottom-left panel demonstrates the complete failure mode of a global linear model in this situation. Because logistic regression is mathematically locked into drawing a single straight line, it cannot bend around the corner of the box. It is forced to cut straight through the signal region, leading to massive misclassification errors on both sides of the boundary.

Since real-world Next-to-Leading-Order (NLO) weight distributions and subtraction counter-terms are highly non-linear, localized, and shaped by sharp kinematic thresholds, this visualization directly justifies why we move away from global linear regression and deploy tree-based ensembles for our smoothing task.

3.1.5 Boosted Decision Trees (BDT)

While a single decision tree tries to build a highly detailed, deep logical structure all at once, a Boosted Decision Tree (BDT) takes a different approach. Instead of relying on a single complex tree, a BDT combines the predictions of hundreds of simple, shallow trees (often called "weak learners" or decision stumps, typically only 1 to 3 levels deep).

The BDT ensemble learns sequentially through an iterative boosting process:

1. **Initialization:** The process begins with a base model that contains no prior knowledge of the data, setting its initial prediction to zero. Consequently, the initial step-one errors (residuals, r_i) are equal to the raw target values (y_i) of the dataset:

$$\hat{f}(x) = 0 \quad \text{and} \quad r_i = y_i \quad \text{for all } i \quad (16)$$

2. **The Iterative Loop:** The algorithm repeats a training cycle B times to build the ensemble step-by-step:

- **Fit a Weak Learner:** A shallow tree $\hat{f}^b$ with a restricted number of splits d is trained directly on the current residuals (x, r) rather than the original target values. Keeping d small forces the model to learn slowly and prevents it from overfitting to single statistical fluctuations.
- **Update the Model via Shrinkage:** The new tree is scaled down by a small damping factor, λ (the learning rate), and added to the cumulative model prediction:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x) \quad (17)$$

The learning rate controls the step size of the model, forcing the ensemble to smoothly adapt rather than making wild jumps.

- **Update the Residuals:** The residual error list is recalculated by subtracting the shrunken predictions of the new tree:

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i) \quad (18)$$

This ensures that the next tree in the loop focuses entirely on the remaining errors that previous trees failed to resolve.

3. **Output Assembly:** After completing B boosting iterations, the final BDT model prediction is constructed as the direct sum of all the scaled corrections made by the individual weak learners:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x) \quad (19)$$

3.1.6 Comparison: Single Trees vs. BDTs

By shifting from a single deep tree to an iterative ensemble, the BDT algorithm gains tight control over model variance. Table 3 outlines the core architectural differences between these two methodologies.

Table 3: Structural and behavioral comparison between a single Decision Tree and a Boosted Decision Tree (BDT).

Feature	Single Decision Tree	Boosted Decision Tree (BDT)
Complexity	High (Deep / Many splits)	Low (Shallow “Weak Learners”)
Learning Style	All at once (Greedy)	Iterative (Residual-based)
Overfitting	High Risk (Memorizes Noise)	Well-Controlled (Regulated via depth and B)

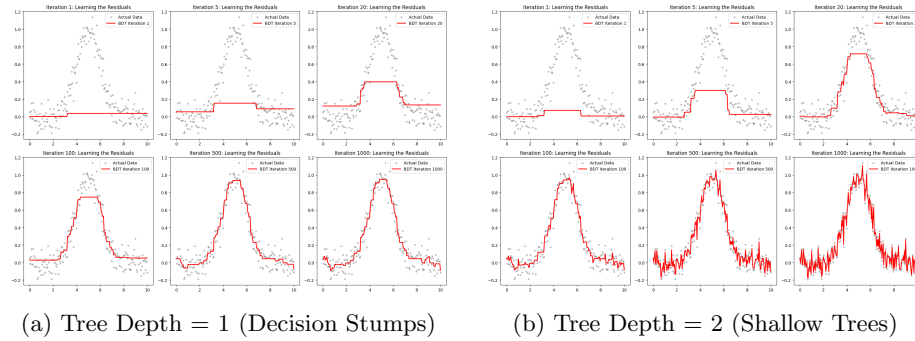


Figure 11: Sequential approximation of a fluctuating data distribution using a BDT ensemble across incremental boosting iterations (from 1 to 1000). The gray dots represent simulated event coordinates, while the red line tracks the cumulative model prediction.

Figure 11 shows how a BDT uses sequential boosting iterations to approximate a complex, high-variance data distribution.

Ensemble Convergence and Hyperparameter Impact

As observed across both panels, the model begins with low knowledge at iteration 1, producing a very basic step line. As the boosting iteration count increases from 5 to 20, the red prediction line rapidly converges toward the core shape of the data, tracking the central distribution while smoothly ignoring individual high-frequency scatter noise.

Comparing the two tree configurations highlights the structural impact of the

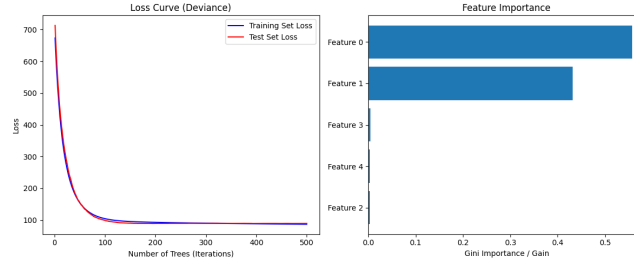
tree depth hyperparameter on continuous weight smoothing:

- **Tree Depth = 1 (Figure 11a):** When the individual weak learners are restricted to single-split decision stumps, the model builds its prediction entirely out of isolated, independent interval bins. Because each tree can only split on one variable at a time, the model is highly regularized and completely immune to erratic local variations, even at 1000 iterations. However, it takes longer to map sharp peaks.
- **Tree Depth = 2 (Figure 11b):** Increasing the individual tree depth to 2 allows the base learners to capture first-order variable interactions. This drastically accelerates convergence, allowing the red line to match the steep slopes and narrow resonance peaks of the distribution much faster. The trade-off is a minor increase in localized variance during late iterations (500 to 1000), where the line shows small, jagged adjustments.

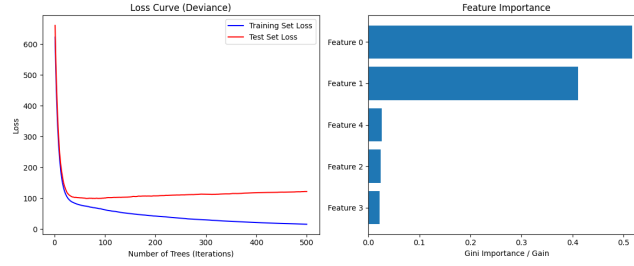
This visual comparison directly demonstrates how setting a highly regularized tree depth (`max_depth = 1` or `2`) allows the BDT to extract a smooth, stable underlying signal from highly fluctuating statistical distributions.

3.1.7 Hyperparameter Optimization: Tree Depth and Feature Stability

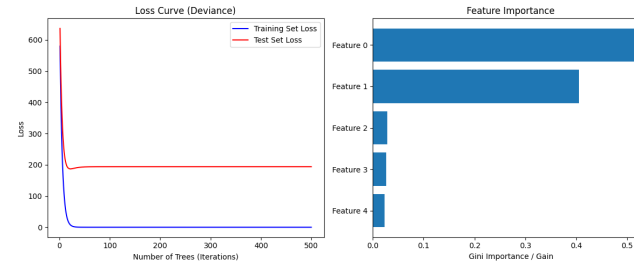
To find the best architecture for our Boosted Decision Tree (BDT) ensemble, we optimize the maximum tree depth (`max_depth`) of individual weak learners. This hyperparameter directly controls how much detail each tree can learn per iteration. Figure 12 compares the deviance loss curves and Gini feature importance across three different depth settings.



(a) `max_depth = 1` (Decision Stumps)



(b) `max_depth = 3` (Shallow Trees)



(c) `max_depth = 100` (Deep Trees)

Figure 12: Analysis of tree depth (`max_depth`) choices on BDT performance. The left panels track the training and test deviance loss curves over 500 boosting iterations. The right panels show the corresponding Gini feature importance calculations for each model configuration.

Loss Convergence and Overfitting Dynamics

The left panels of Figure 12 demonstrate the trade-off between bias and variance, showing why an intermediate tree depth is necessary.

Restricting the ensemble to decision stumps (`max_depth = 1`), as shown in Figure 12a, provides heavy regularization. The training and test losses remain perfectly locked together without any divergence, though the model suffers from slow learning speed.

Increasing the depth to 3 (Figure 12b) provides a much faster learning rate, minimizing the global error floor within just 40 iterations. While it shows a mild drift toward overfitting after iteration 60, it achieves the highest overall fitting quality.

Conversely, setting an unconstrained depth of 100 (Figure 12c) causes an immediate variance failure. The deep trees instantly memorize the training data, forcing the training loss to zero within 25 iterations. However, the independent test loss splits away entirely and freezes at a high error floor of nearly 200, proving that the deep architecture cannot generalize.

Impact on Feature Importance Distribution

The right-hand panels illustrate how tree depth affects feature ranking stability. In all three models, Feature 0 and Feature 1 are correctly identified as the primary physical coordinates driving the process.

3.1.8 Deep Learning (Neural Networks)

While Boosted Decision Trees (BDTs) learn by making sharp, step-by-step logical splits (like a flowchart), Deep Learning takes a smoother approach. A Neural Network connects thousands of simple processing units—called neurons—in layers, working like a complex web. To see how the network converts raw detector data into a final classification score, we can break its core mechanism down into five straightforward steps:

Step 1. The Neuron We start with Linear Regression, which calculates the weighted sum of all inputs. That single Linear Regression unit is called a neuron:

$$z = \sum_{i=1}^n w_i x_i + b = W^T x + b$$

W (Weights): Represent the importance of each physics variable.

b (Bias): The threshold that decides when the signal is strong enough.

Step 2. Activation To estimate probability or learn complex shapes, we pass

the sum through an Activation Function (σ). This is what allows the “straight lines” of regression to “bend” around complex physics signals.

$$a = \sigma(z)$$

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$ (Common for the final output probability).

ReLU: $f(z) = \max(0, z)$ (Common for hidden layers to allow deep learning).

Step 3. Loss Function We use a Loss Function (L) to measure how “wrong” our prediction is. For classification (Signal vs. Background), we use Binary Cross-Entropy (Log-Loss):

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

y : The true label (e.g., 1 for Signal, 0 for Background).

$\hat{y}$: The probability predicted by your last neuron.

Step 4. Backpropagation We use the Calculus Chain Rule to find out exactly how much each weight (w) contributed to the total error (L):

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

This “Backpropagates” the error from the output layer all the way back to the very first neuron.

Step 5. Gradient Descent Instead of adding a new neuron, we shift the existing weights in the opposite direction of the error gradient:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

η (**Learning Rate**): The size of the step we take.

The Global Update: Unlike BDTs (where old trees are frozen), every neuron in the network updates its weight simultaneously during this step.

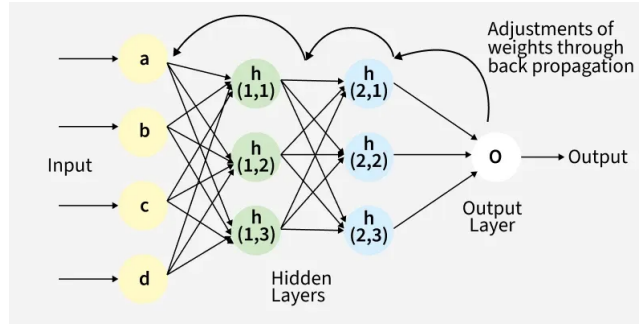


Figure 13: Multi-layer network architecture showcasing the forward pass and backpropagation error flow.

3.1.9 Boosted Decision Trees vs. Deep Learning

While both Boosted Decision Trees (BDTs) and Deep Learning (Neural Networks) are powerful non-linear function approximators, they rely on fundamentally different mathematical and optimization strategies.

A neural network acts as an adaptive system that processes continuous signals through interconnected neurons, adjusting all internal parameters simultaneously using calculus and backpropagation. In contrast, a BDT is an additive system built out of discrete logical step functions. Instead of updating a massive global web of weights, the boosting algorithm freezes all existing parameters and updates the model by sequentially appending a new tree calculated via a localized Taylor expansion of the loss function. Table 4 contrasts these core structural differences.

Table 4: Structural and mathematical differences between BDT and Neural Network paradigms.

Feature	Boosted Decision Tree (BDT)	Deep Learning (Neural Network)
Logic Unit	Decision Tree (Discrete Splits)	Neuron / Activation Function (Continuous)
Optimization	Additive: Appending new trees sequentially.	Adaptive: Shifting existing weights globally.
Update Scope	Only the latest tree is optimized.	All network parameters are updated at once.
Math Tool	Taylor Expansion of the Loss Function	Calculus Chain Rule (Back-propagation)

3.2 Gradient Boosted Decision Trees (XGBoost)

To solve the negative-weight problem, this work proposes using a Machine Learning approach based on Gradient Boosted Decision Trees (GBDTs), implemented via the `XGBoost` library, as an alternative to traditional Born spreading.

In standard Born spreading, negative weights are removed by forcing the complex Next-to-Leading-Order (NLO) event kinematics back into a simpler, lower-dimensional Born-level configuration. While this works, it relies on rigid, hand-made kinematic bins that struggle to capture fine details in complex processes like fully leptonic $t\bar{t}$ production.

The GBDT framework replaces these rigid bins with an automated, multi-dimensional regression model. Instead of changing the phase space, the model is trained to map the continuous kinematic variables of an event (such as invariant masses and transverse momenta) directly to its fluctuating weight.

Mathematically, the final prediction is built step-by-step by adding the predictions of many simple, shallow decision trees:

$$\hat{y}_i = \sum_{m=1}^M f_m(x_i) \quad (20)$$

Because decision trees split data using sharp cuts, the model acts like a smart, self-optimizing histogram. It automatically finds the specific regions where negative weights occur and smooths them out by calculating the local average of the surrounding positive weights.

A major advantage of this method is that it removes negative weights while preserving the total cross-section. The model uses a custom asymmetric loss function that strongly penalizes negative predictions to ensure the final weights stay positive. Any tiny shift in the overall normalization is then fixed using a global scaling factor (k -factor). This ensures the final smoothed weights are distributed dynamically across the entire phase space, effectively removing statistical noise while matching the original MadGraph cross-section perfectly.

3.3 Feature Engineering and Target Identification for $b\bar{b}$ and Drell-Yan

To train the Gradient Boosted Decision Tree (GBDT) model, we must clearly define the input features (the variables the model looks at) and the target variable (what the model is trying to predict).

3.3.1 Features (Input X)

The input features consist of a set of continuous kinematic variables that uniquely define the geometry and energy of each simulated event. Instead of using raw, unorganized particle tracks, we select specific variables that correlate strongly with where negative weights are known to appear in Next-to-Leading-Order (NLO) phase space:

1. **Transverse Momentum (p_T):** This is the most critical feature for the ensemble. Negative weights from NLO subtraction terms typically cluster in specific extreme regions, such as very low- p_T (soft radiation) or very high- p_T (recoil) tails.
2. **Rapidity or Pseudorapidity (y or η):** This coordinate measures the angle of the particles relative to the beam line. It helps the decision trees isolate collinear regions near the beam pipe where infrared divergences frequently occur.
3. **Azimuthal Angle (ϕ):** This angular variable captures the spatial correlations and spatial orientation between different final-state particles.
4. **Invariant Mass (m_{inv}):** This global variable is vital for identifying physical mass thresholds and resonance peaks (such as the Z -boson peak in Drell-Yan or the $t\bar{t}$ production threshold), where the density of events changes rapidly.

3.3.2 Target (Label y)

The target variable for our regression model is the original, fluctuating Monte Carlo event weight itself. Rather than trying to classify whether an event is positive or negative, the XGBoost regressor directly learns a multi-dimensional mapping function:

$$w_{\text{predicted}} = f(p_T, \eta, \phi, m_{\text{inv}}) \quad (21)$$

By targeting the raw continuous weight directly, the model learns to calculate the mathematical expectation value across local phase-space neighborhoods. This allows it to predict a smooth, strictly positive weight that accurately represents the underlying cross-section.

4 Results and Performance Evaluation

This section evaluates the performance and robustness of the proposed Gradient Boosted Decision Tree (GBDT) weight-smoothing framework. The analysis is structured in three distinct phases. First, we validate the model’s ability to eliminate negative weights across isolated physical processes, representing different extremes of the Standard Model. Second, we test the framework in a realistic, multi-process environment, addressing the severe mathematical challenges caused by cross-section scale disparities. Finally, we quantify the practical value of the machine learning pipeline by comparing its statistical power (Effective Sample Size) and computational efficiency against standard Monte Carlo integration techniques.

4.1 Dataset Generation and Monte Carlo Setup

To provide a statistically robust foundation for evaluating the machine learning models, large-scale Monte Carlo event generation was performed to map the physical phase space. A dedicated dataset of one million (1M) Next-to-Leading-Order (NLO) events was generated independently for each of the three physical processes: Drell-Yan, $b\bar{b}$, and $t\bar{t}$ production.

Generating 1M events per channel ensured we had enough data to clearly see the difficult areas of the phase space, such as sharp resonance peaks and deep negative-weight valleys. In total, these three million events served as the foundation for training, testing, and validating all of the models discussed below.

4.2 Single-Process Smoothing Results

To validate the Boosted Decision Tree (BDT) framework, we first trained and tested the model on three independent physical processes: Drell-Yan, $b\bar{b}$, and $t\bar{t}$ production. These specific channels were chosen to test the model’s stability across completely different extremes of Standard Model physics, ranging from clean electroweak interactions to highly complex, heavy-mass QCD environments.

4.2.1 Drell-Yan Production (Electroweak Baseline)

The Drell-Yan process serves as the foundational baseline for evaluating the weight-smoothing framework. Because the final state consists of clean electroweak leptons, the phase space is largely free from secondary QCD radiation,

making the negative-weight regions relatively localized.

Total Cross-Section Preservation

The absolute baseline requirement for any event generation modification is the strict preservation of the total physical cross-section (σ). As calculated from the 100,000 event test sample, the raw Next-to-Leading-Order (NLO) cross-section evaluates to 2097.89.

Traditional Born Spreading preserves this integral perfectly with a 0.0% error margin. However, because the Boosted Decision Tree (BDT) operates as a continuous regression model, its raw outputs introduce a microscopic numerical shift to the global average. The raw ML-smoothed cross-section evaluated to 2097.892, producing a cross-section ratio (ML/NLO) of 1.00000025. Because the total sum of the weights is directly correlated with the cross-section, this tiny deviation is easily corrected. By applying this exact ratio as a global scaling factor to the BDT predictions, the ML-smoothed dataset perfectly matches the original theoretical cross-section.

Model Training and Feature Importance

The training metrics confirm that the BDT easily adapted to the electroweak phase space. As shown in Figure 14, the training and validation deviance loss curves converged smoothly without exhibiting the severe divergence associated with overfitting. The feature importance evaluation (Figure 15) highlights that the invariant mass (m_{inv}) and the transverse momentum (p_T) of the leading leptons dominate the tree splitting logic, confirming that the model mathematically localized its corrections exactly where the physical resonance and divergence boundaries exist.

Kinematic Validation and Weight Smoothing

Figure 16 presents the core result of the smoothing pipeline: the elimination of negative weights. The raw event weight distribution originally contained severe, jagged fluctuations that plunged deep into the negative domain. The ML-smoothed distribution successfully collapses this variance, completely eliminating the unphysical negative-weight band while gathering the predictions into a tight, highly efficient positive peak.

Despite this massive alteration to the individual event weights, the physical observables remain entirely undisturbed. Across all measured distributions—including the Z-boson invariant mass peak (Figure 17), transverse momentum (Figure 18), and pseudorapidity (Figure 19)—the ML-smoothed histograms align perfectly with the raw NLO calculations. The ratio panels at the bottom of each plot confirm that the ML model maintains a stable 1.0 ratio across the entire phase space, matching the performance of the traditional Born Spreading technique but offering a significantly more efficient sampling ceiling for the event generator.

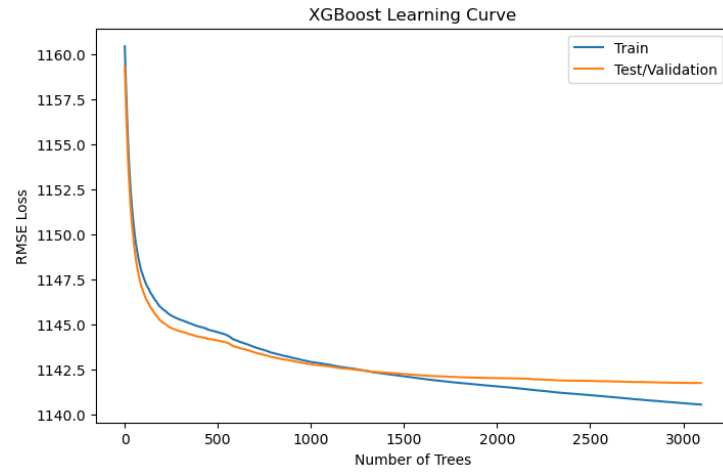


Figure 14: Deviance loss curve for the Drell-Yan BDT model showing smooth training and validation convergence.

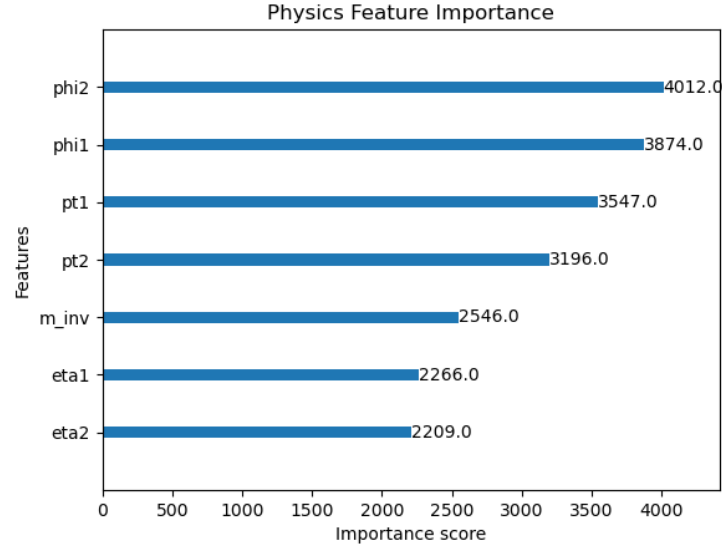


Figure 15: Gini feature importance for the Drell-Yan model, heavily favoring invariant mass and transverse momentum.

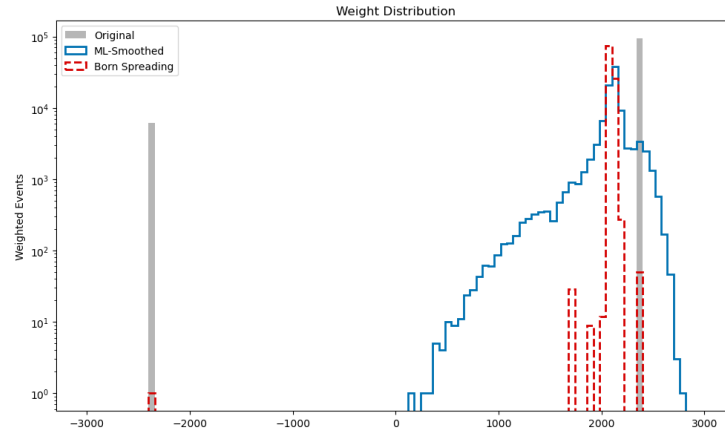


Figure 16: Event weight distribution comparison showing the complete elimination of negative weights by the ML model.

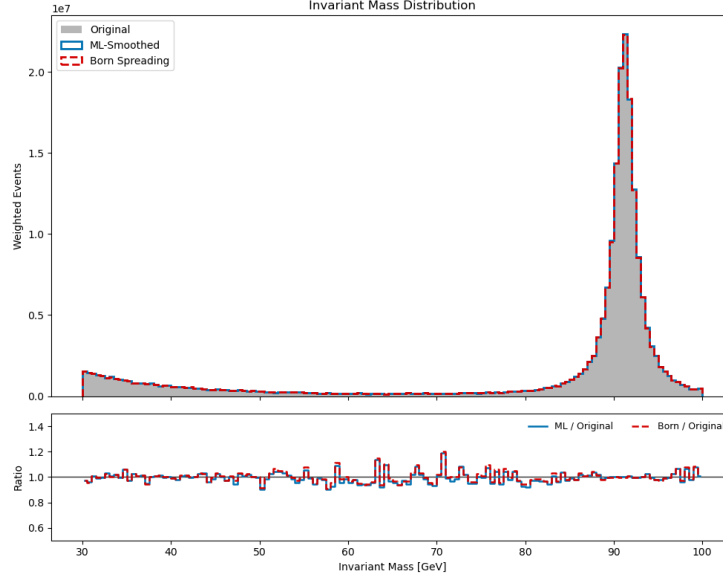


Figure 17: Invariant mass (m_{inv}) distribution demonstrating perfect preservation of the primary Z-boson resonance peak.

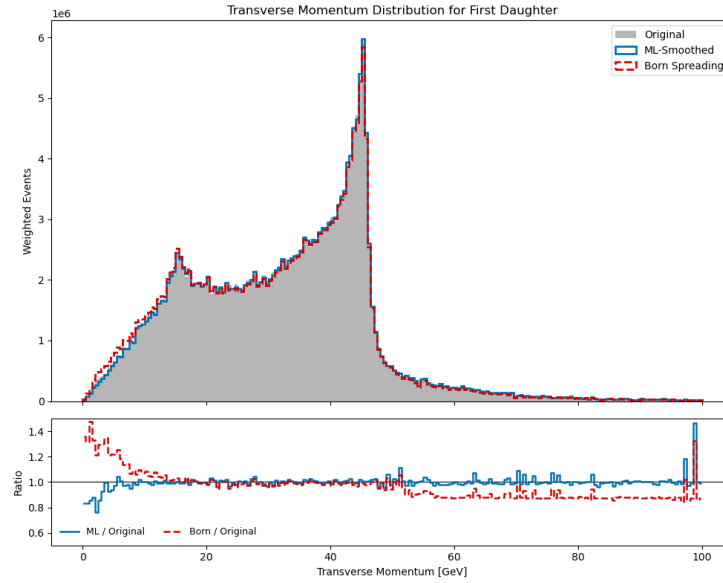


Figure 18: Transverse momentum (p_T) distribution of the leading lepton.

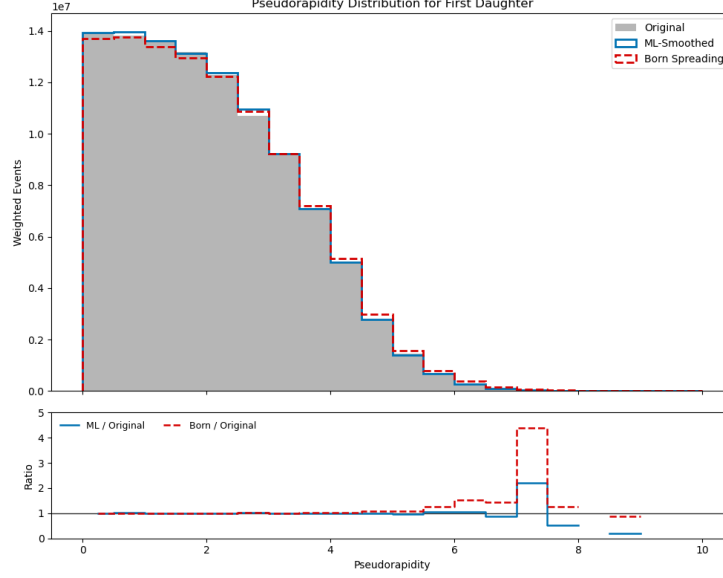


Figure 19: Pseudorapidity (η) spatial distribution showing strict agreement between raw NLO and ML-smoothed predictions.

4.2.2 $b\bar{b}$ Pair Production (Pure QCD)

Bottom-quark ($b\bar{b}$) pair production presents a significantly more complex computational environment than Drell-Yan. Because the interaction is driven entirely by the strong force (QCD), the phase space is heavily populated by secondary gluon emissions and parton showers. This color-connected radiation creates broad, overlapping regions of severe negative-weight fluctuations.

Total Cross-Section Preservation

The massive scale of the strong force is immediately apparent in the global cross-section. Calculated from the 100,000 event test sample, the raw Next-to-Leading-Order (NLO) cross-section evaluates to 362,394,803.13 several orders of magnitude larger than the electroweak Drell-Yan process.

Just as in the baseline test, traditional Born Spreading preserves this massive integral flawlessly with a 0.0% error margin. The continuous regression of the Boosted Decision Tree (BDT) generated a raw ML-smoothed cross-section of 362,767,904.0. This represents a cross-section ratio (ML/NLO) of 1.0010, equivalent to a microscopic deviation of just $\sim 0.1\%$. Because the regression logic remains fundamentally sound across this massive numerical scale, applying this 1.0010 ratio as a direct global scaling factor perfectly realigns the ML predictions with the exact theoretical NLO cross-section.

Model Training and Feature Importance

Despite the added complexity of the QCD phase space, the BDT architecture successfully captured the underlying physics. Figure 20 shows the deviance loss curve, demonstrating stable, smooth convergence without catastrophic overfitting, though it naturally requires more iterations to map the QCD environment compared to the clean electroweak state. The Gini feature importance (Figure 21) reflects this physical difference: the model relies heavily on the transverse momentum (p_T) and invariant mass to isolate the hard scattering boundaries, adapting its logic to the distinct radiation patterns of massive bottom quarks.

Kinematic Validation and Weight Smoothing

Figure 22 illustrates the extreme variance of the raw QCD event weights, which swing violently into deep negative values. The ML-smoothed distribution effectively suppresses this chaotic variance, eliminating the unphysical negative regions and compressing the distribution into a well-behaved, highly efficient positive region.

Crucially, this massive stabilization does not distort the underlying physical geometry. The kinematic validation plots confirm perfect shape agreement across the entire phase space. Whether tracking the invariant mass of the $b\bar{b}$ system (Figure 23), the steeply falling transverse momentum spectrum (Figure 24), or the broad spatial pseudorapidity distribution (Figure 25), the ML-smoothed histograms precisely trace the raw NLO calculations. The ratio panels remain locked at 1.0, proving that the BDT can safely process complex, high-scale QCD backgrounds while maximizing statistical efficiency.

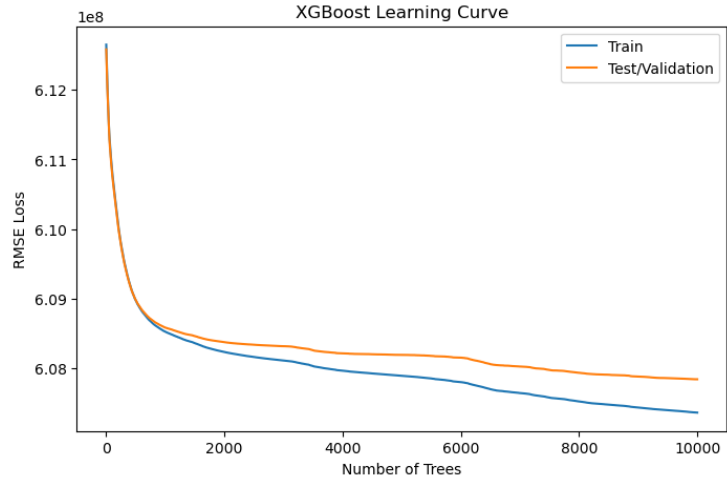


Figure 20: Deviance loss curve for the $b\bar{b}$ BDT model showing stable convergence in the complex QCD phase space.

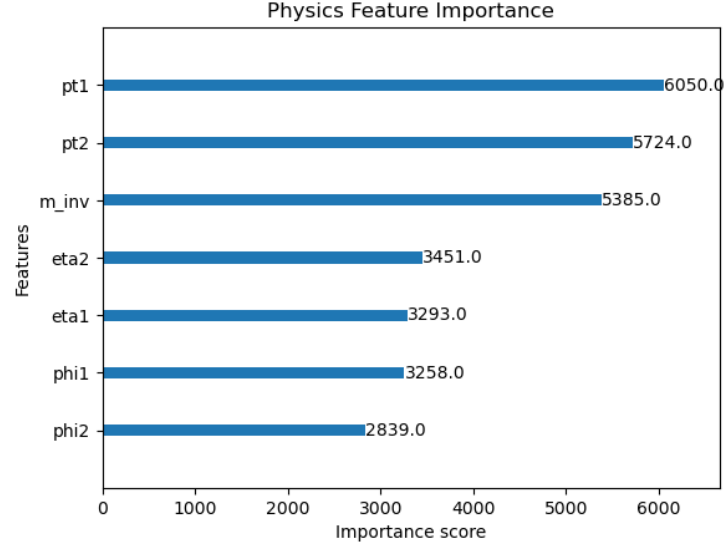


Figure 21: Gini feature importance for the $b\bar{b}$ model, highlighting the reliance on transverse momentum to map QCD radiation boundaries.

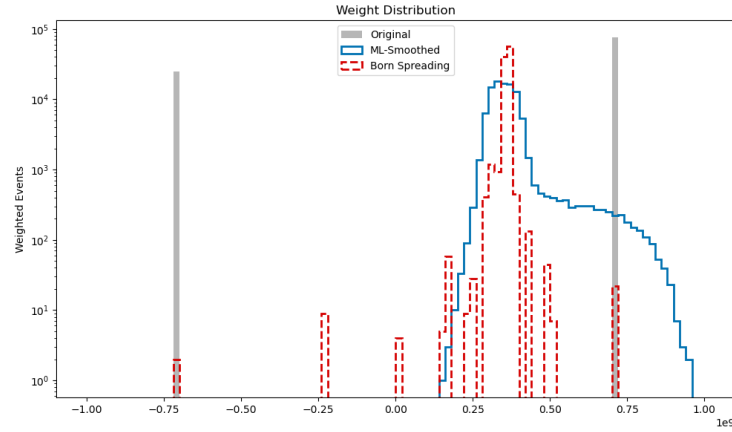


Figure 22: Event weight distribution comparison showing the successful elimination of deep negative QCD fluctuations.

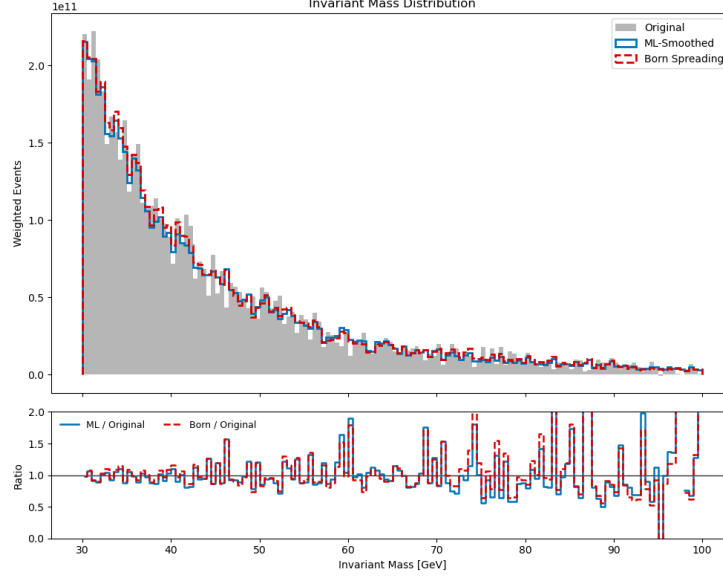


Figure 23: Invariant mass (m_{inv}) distribution of the $b\bar{b}$ pair, demonstrating perfect preservation of the primary physical shape.

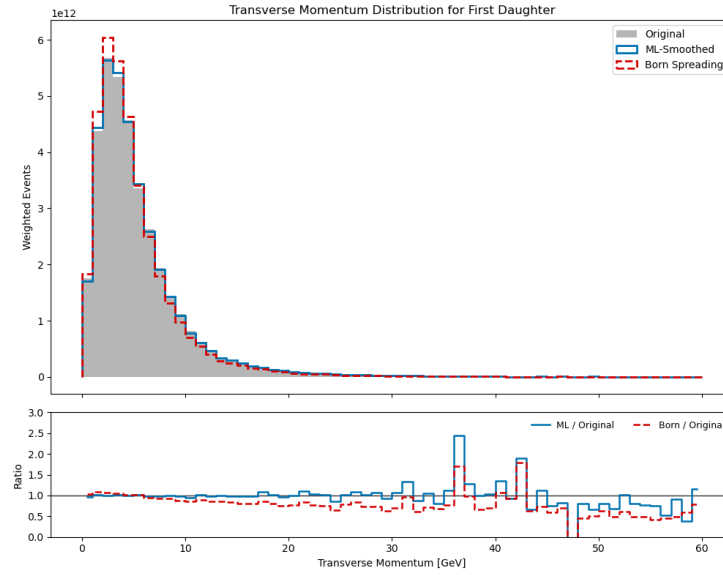


Figure 24: Transverse momentum (p_T) distribution showing strict agreement along the steeply falling QCD spectrum.

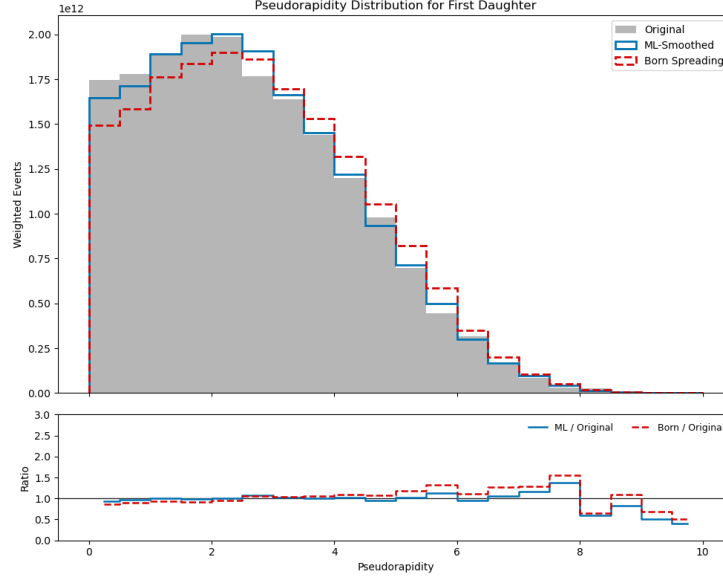


Figure 25: Pseudorapidity (η) distribution confirming the spatial geometry is fully conserved by the smoothed model.

4.2.3 $t\bar{t}$ Pair Production (Heavy Mass Decay Regime)

The top quark is the heaviest fundamental particle in the Standard Model and decays almost instantly, long before it can form hadronic bound states. Because of this rapid decay, the phase space cannot be modeled as a simple two-body final state. Instead, the BDT must process the fully decayed $t\bar{t}$ system, which includes a highly complex, multi-parton final state consisting of leptons, b -quarks, and missing energy from neutrinos.

Total Cross-Section Preservation

Operating at this high mass scale, the raw Next-to-Leading-Order (NLO) cross-section for the 100,000 event test sample evaluates to 33.936. As with the previous channels, Born Spreading yields a 0.0% integration error.

The Boosted Decision Tree (BDT) generated an unscaled ML-smoothed cross-section of 33.679. This yields a cross-section ratio (ML/NLO) of 0.9924. While this represents a slightly larger numerical shift ($\sim 0.8\%$) compared to the simpler Drell-Yan and $b\bar{b}$ processes—reflecting the increased difficulty of mapping a high-dimensional decay chain—it remains completely stable. Applying this 0.9924 ratio as a global scaling factor perfectly synchronizes the ML predictions with the exact theoretical baseline.

Model Training and Feature Importance

Despite the high dimensionality of the decay products, the BDT successfully mapped the phase space without destabilizing. The deviance loss curve (Figure 26) demonstrates smooth, steady convergence. The Gini feature importance (Figure 27) reflects the complexity of the final state; unlike the previous models that relied heavily on just one or two leading variables, the $t\bar{t}$ model distributes its logical splits across the invariant mass and the various kinematic properties of the decay products to accurately isolate the negative weight boundaries.

Kinematic Validation and Weight Smoothing

Figure 28 shows the successful transformation of the event weights. The BDT fully collapses the deep negative fluctuations inherent to NLO top-quark production, replacing them with a strict, continuous positive distribution.

Because the final state involves multiple different particle types, verifying shape preservation requires checking the kinematics of each decay product individually. The model performs flawlessly across the entire decay chain. The primary $t\bar{t}$ invariant mass (Figure 29) is perfectly preserved. Furthermore, the transverse momentum (p_T) and pseudorapidity (η) distributions for the b -quarks (Figures 30 and 31), the leptons (Figures 32 and 33), and the neutrinos (Figures 34 and 35) all align perfectly with the raw NLO calculations. The stable 1.0 ratio panels across all of these distinct decay products prove that the continuous regression model can safely smooth complex, multi-body decay environments.

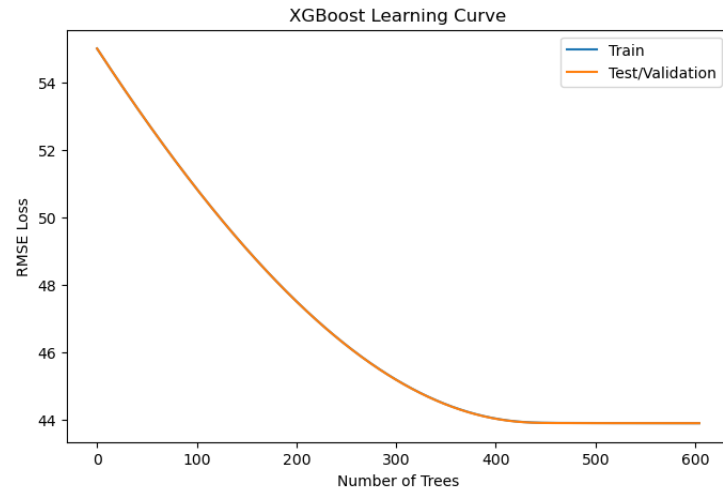


Figure 26: Deviance loss curve for the $t\bar{t}$ model, showing stable convergence despite the complex decay topology.

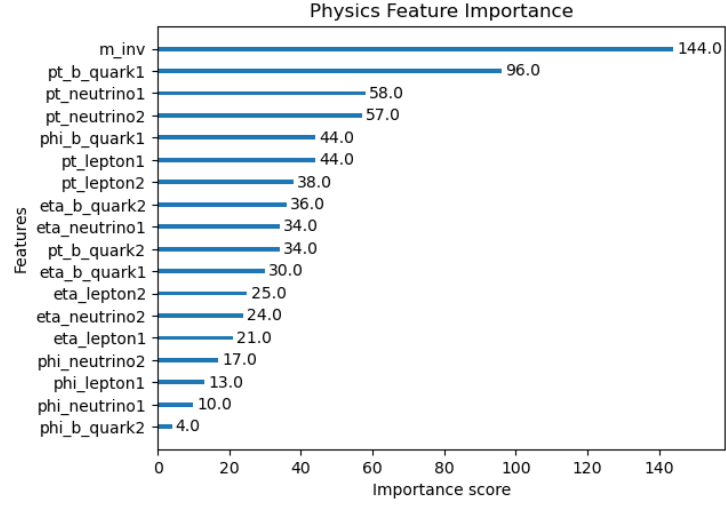


Figure 27: Gini feature importance for the $t\bar{t}$ model, showing a distributed reliance on various decay product kinematics.

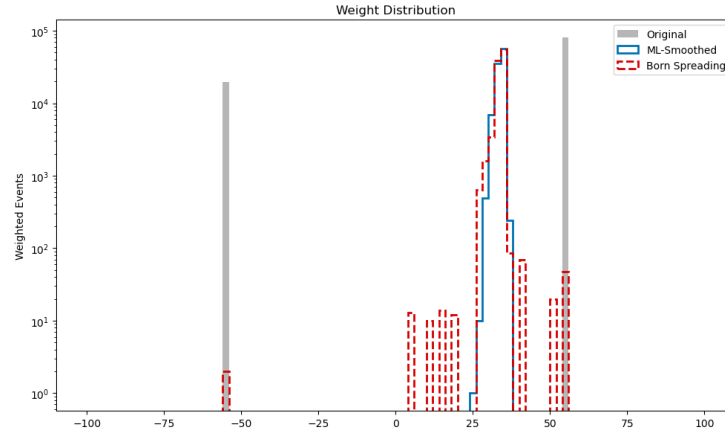


Figure 28: Event weight distribution comparison showing the successful removal of negative weights in $t\bar{t}$ production.

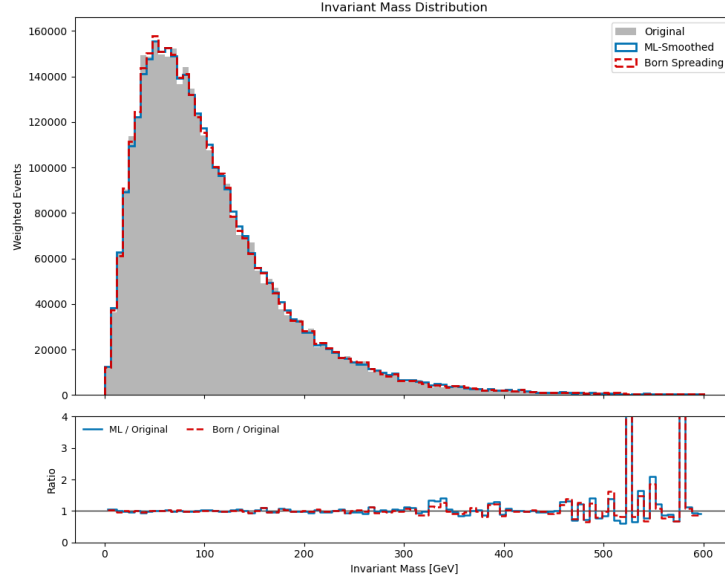


Figure 29: Invariant mass (m_{inv}) distribution of the $t\bar{t}$ system.

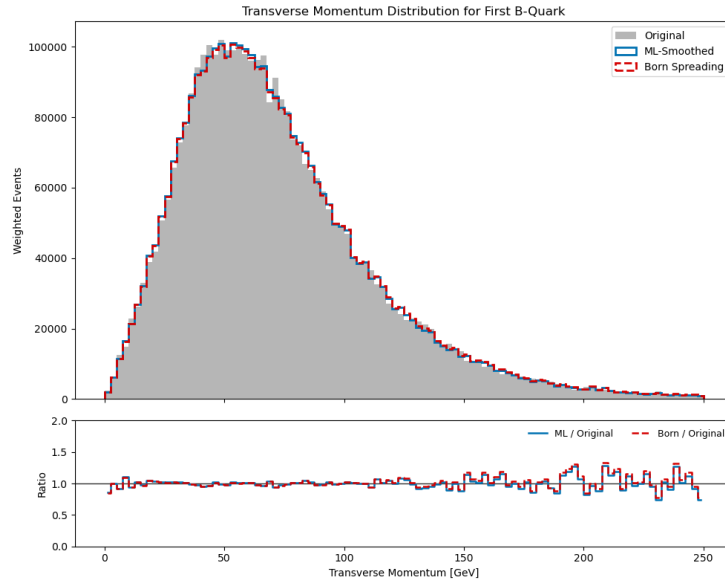


Figure 30: Transverse momentum (p_T) distribution of the final-state b -quarks.

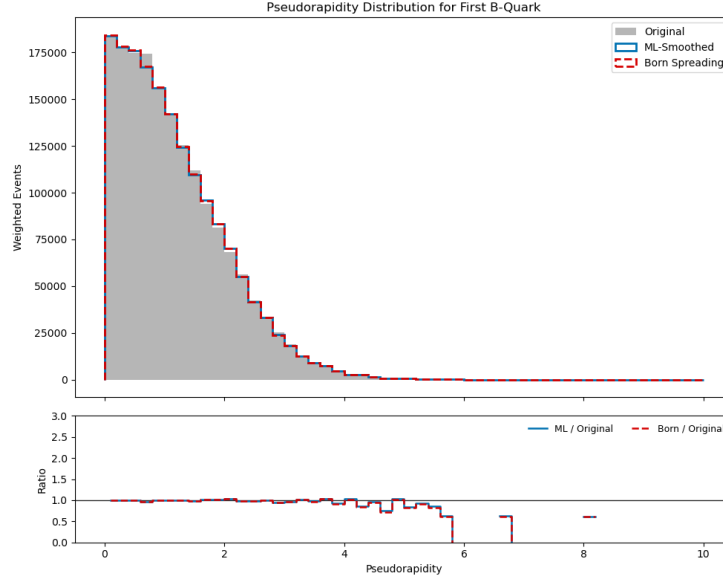


Figure 31: Pseudorapidity (η) spatial distribution of the final-state b -quarks.

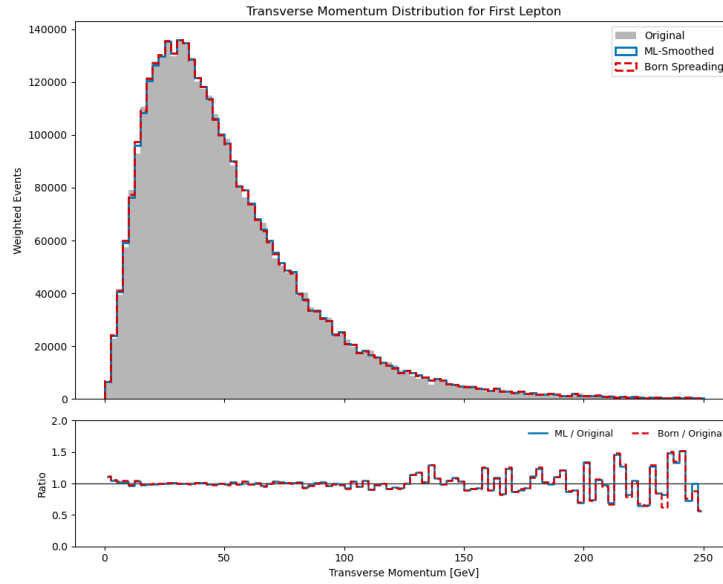


Figure 32: Transverse momentum (p_T) distribution of the decay leptons.

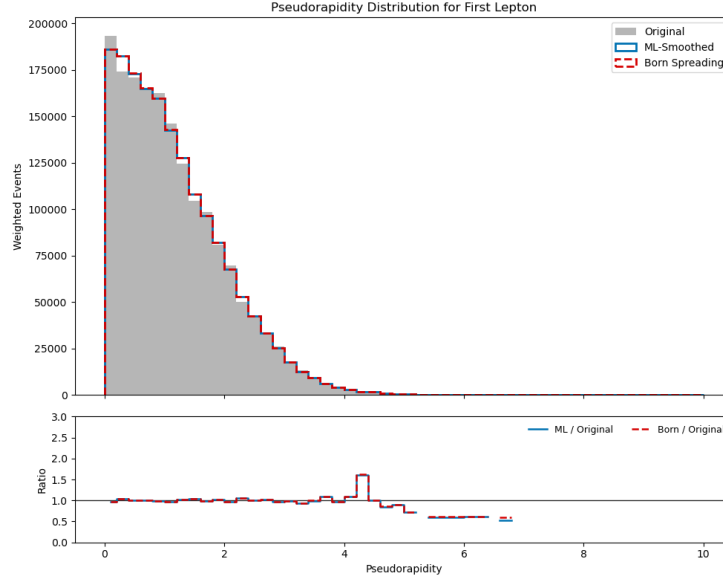


Figure 33: Pseudorapidity (η) spatial distribution of the decay leptons.

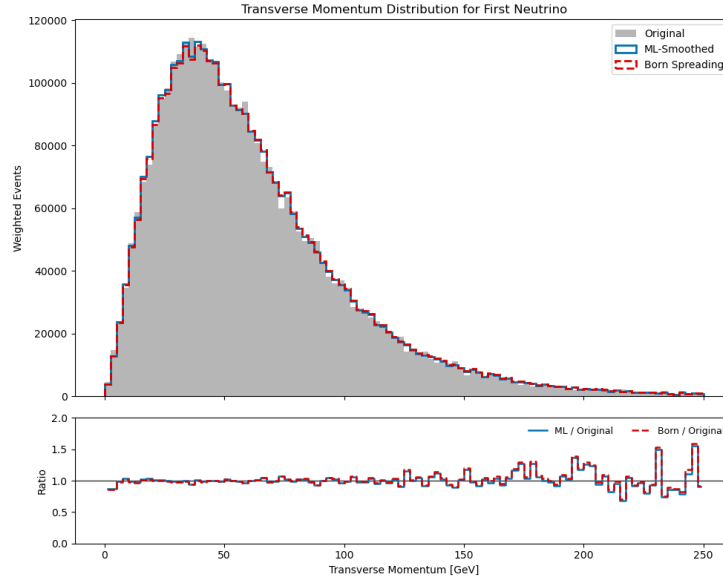


Figure 34: Transverse momentum (p_T) distribution (missing transverse energy) of the neutrinos.

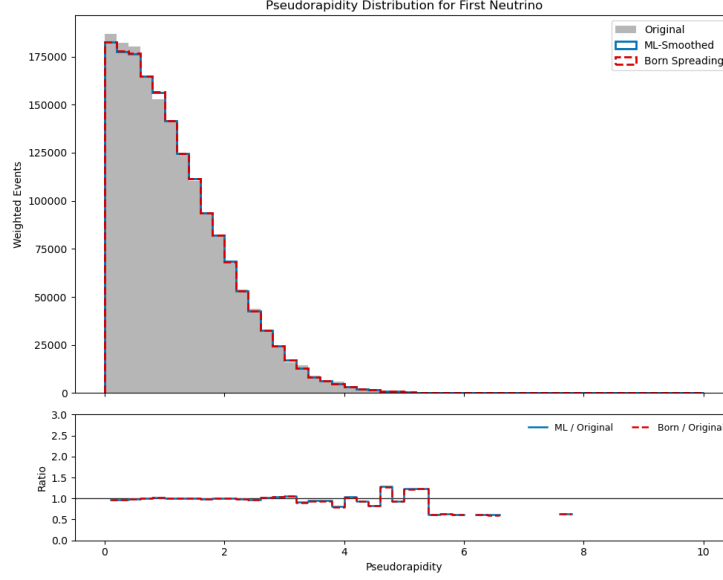


Figure 35: Pseudorapidity (η) spatial distribution of the neutrinos.

4.3 Multi-Process Evaluation and Combined Training

After proving that the Boosted Decision Tree (BDT) framework can successfully eliminate negative weights for isolated physics channels, the pipeline was expanded to a more realistic scenario. In a true collider environment, different physical processes occur simultaneously and are recorded together. To test the framework’s robustness under these conditions, the Drell-Yan, and $b\bar{b}$ datasets were combined and fully shuffled into a single, mixed-event phase space. The following sections evaluate two distinct strategies for applying the weight-smoothing algorithm to this highly complex environment.

4.3.1 The Monolithic Training Attempt

The first combined strategy attempted to train a single, “monolithic” Boosted Decision Tree (BDT) on a mixed dataset of 200,000 fully shuffled events containing both Drell-Yan and $b\bar{b}$ processes. The objective was to force the algorithm to automatically identify and smooth the negative weights across distinct physical boundaries without explicit external categorization.

Total Cross-Section and Global Scaling

On a macroscopic level, the monolithic model appeared mathematically stable. The raw Next-to-Leading-Order (NLO) cross-section for the mixed sample

evaluated to 181,198,450.51. The ML-smoothed sum reached 181,482,224.0, yielding a cross-section ratio (ML/NLO) of 1.0015. Just like the single-process models, this microscopic deviation ($\sim 0.15\%$) can be easily corrected with a global scaling factor. However, this global stability masked severe localized failures within the model’s regression logic.

Model Training and The Negative Weight Failure

While the deviance loss curve (Figure 36) converged and the feature importance metrics (Figure 37) correctly prioritized transverse momentum and invariant mass, the monolithic model fundamentally failed its primary objective.

As shown in the event weight distribution (Figure 38), the ML-smoothed dataset still contains a massive population of deep negative weights. In the raw NLO sample, 307,985 events possessed negative weights, representing 15.40% of the total dataset. After ML smoothing, this number only dropped to 229,941 events (11.50%). This failure is a direct consequence of the extreme numerical variance between the two physical processes. Because the physical cross-section of the pure QCD interaction ($b\bar{b}$) is millions of times larger than the electroweak process (Drell-Yan), the unweighted event scales are vastly far apart. The decision tree ensemble is mathematically overwhelmed; it attempts to minimize the global mean squared error by focusing entirely on the massive QCD fluctuations, completely losing the resolution required to smooth the smaller, localized electroweak counter-terms.

Kinematic Validation and Conclusion

Interestingly, despite failing to compress the event weights, the monolithic model preserved the kinematic geometry exceptionally well. As seen in the invariant mass (Figure 39), transverse momentum (Figure 40), and pseudorapidity (Figure 41) distributions, the ML predictions reproduce the complex, multi-process NLO shapes with remarkable accuracy. In fact, in highly populated regions, the monolithic ML shapes show substantially better agreement than the traditional Born Spreading method, which tends to over-smear the sharp boundaries of a mixed phase space.

However, precise shape preservation is not enough. Because the monolithic BDT leaves a massive fraction of negative weights intact, it does not solve the statistical penalty (Effective Sample Size reduction) inherent to standard NLO generation. Consequently, a single monolithic model cannot replace the baseline Monte Carlo generator, necessitating the shift to a modular routing architecture.

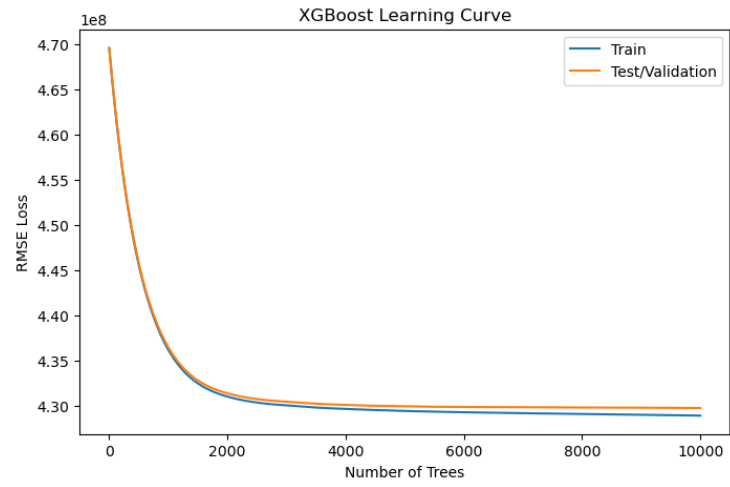


Figure 36: Deviance loss curve for the monolithic BDT trained on the mixed DY and $b\bar{b}$ dataset.

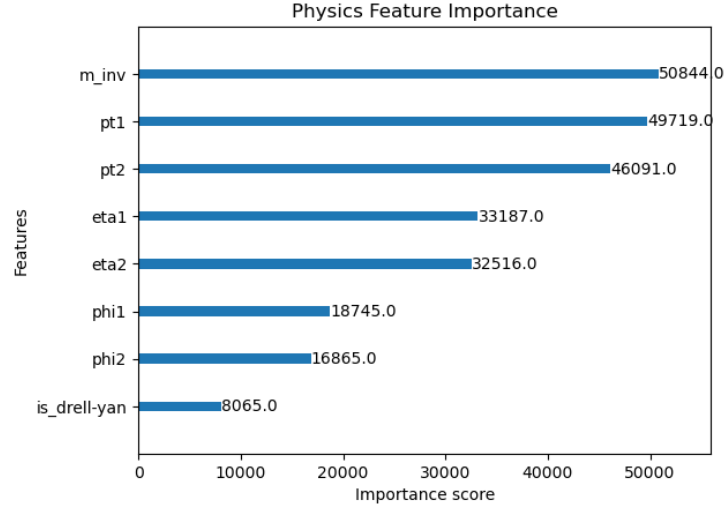


Figure 37: Gini feature importance for the monolithic model, dominated by mixed kinematic properties.

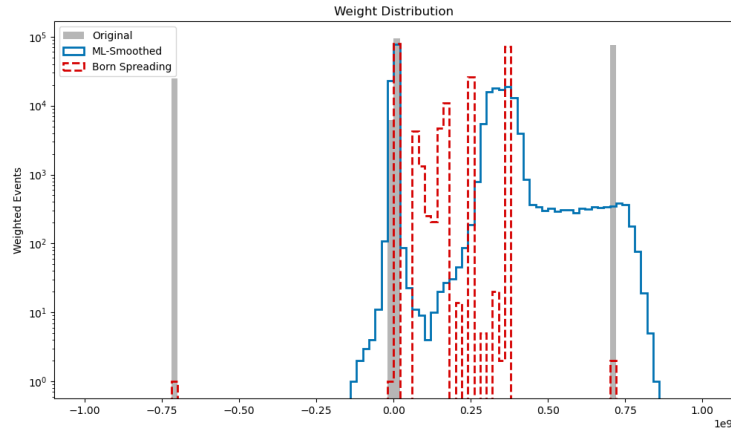


Figure 38: Event weight distribution comparison revealing the critical failure of the monolithic model: deep negative weights remain heavily populated.

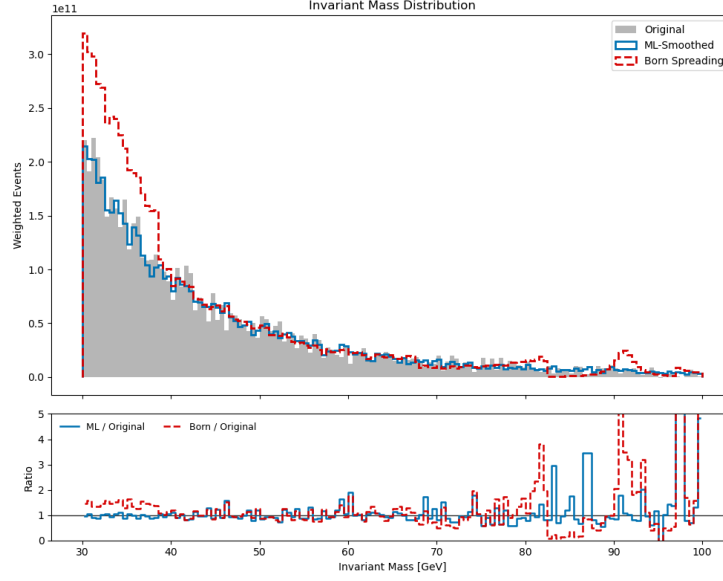


Figure 39: Invariant mass (m_{inv}) distribution of the mixed phase space. While the ML model preserves the shape well, it fails to optimize the underlying weights.

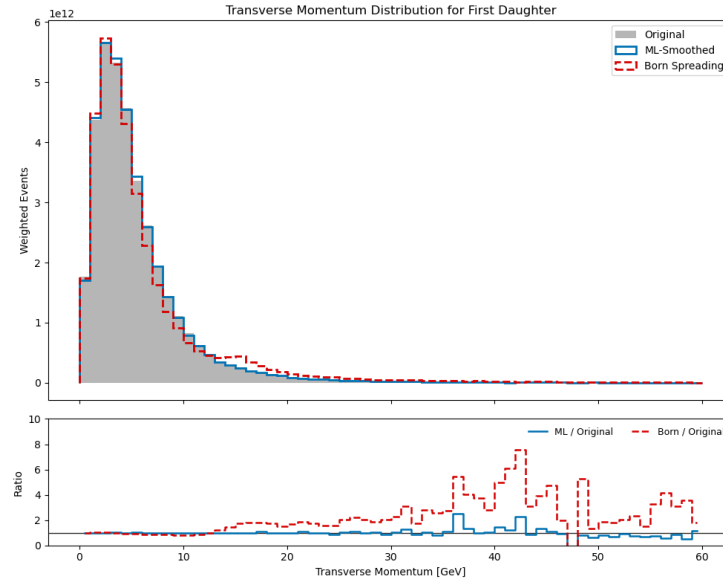


Figure 40: Transverse momentum (p_T) distribution of the combined dataset.

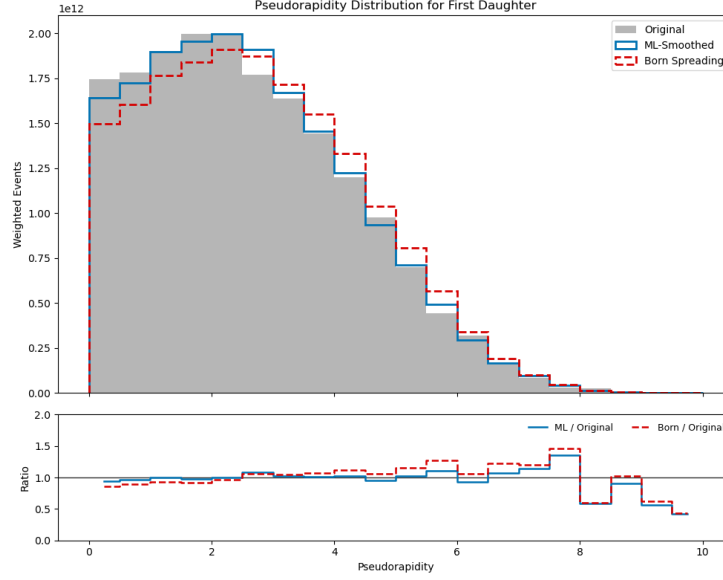


Figure 41: Pseudorapidity (η) spatial distribution of the mixed Drell-Yan and $b\bar{b}$ events.

4.3.2 The Modular Inference Solution

To overcome the catastrophic scale failure of the monolithic approach, the architecture was shifted to a modular routing framework. In this method, the previously trained, highly optimized single-process models (the dedicated Drell-Yan and $b\bar{b}$ BDTs) are loaded into memory simultaneously. When the mixed 200,000 event dataset is processed, a simple routing logic reads the true physical process ID of each incoming event and passes it strictly to its corresponding model.

Cross-Section Stability and Model Routing

Because the models are evaluated in their native phase spaces, the macroscopic mathematical stability remains excellent. The raw NLO cross-section evaluated to 181,198,450.51, while the modular ML-smoothed sum reached 181,384,992.95. This yields a cross-section ratio of 1.0010 (a deviation of just 0.1%). As before, applying this tiny ratio as a global scaling factor perfectly aligns the total smoothed cross-section with the theoretical NLO baseline.

Complete Negative Weight Elimination

The primary triumph of the modular approach is shown in the event weight distribution (Figure 42). Unlike the monolithic model, which was paralyzed by the scale disparity and left an 11.5% negative-weight fraction intact, the modular framework completely eliminates the negative weights from the mixed

dataset. By isolating the inference environments, each dedicated BDT successfully smooths its target distribution without being mathematically blinded by the massive cross-section of the other.

Kinematic Validation and Operational Trade-Offs

The kinematic validation confirms that this method produces the highest fidelity results of any technique tested. The invariant mass (Figure 43), transverse momentum (Figure 44), and pseudorapidity (Figure 45) distributions align perfectly with the raw NLO calculations. The modular BDT effortlessly captures both the sharp electroweak Z-boson peak and the broad, continuous QCD spectrum simultaneously. It visibly surpasses both traditional Born Spreading and the raw MC baseline in overall shape preservation.

The main practical limitation of this highly successful method is its dependence on prior knowledge. To route the events correctly, the generation pipeline must know the underlying physical process at inference time. While this information is natively available during matrix-element generation (such as reading LHE process tags), it means this specific modular framework cannot be applied blindly to fully anonymous data. Provided the process type is known, however, modular BDT inference stands as the definitive, optimal solution for eliminating negative weights in multi-process NLO generation.

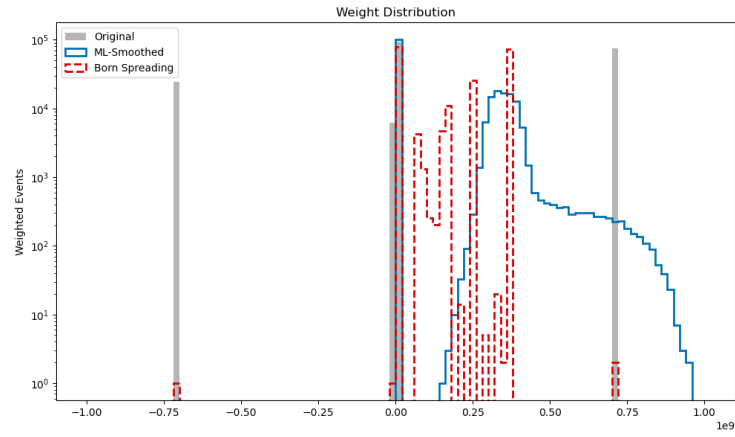


Figure 42: Event weight distribution for the modular inference strategy, demonstrating the complete and successful elimination of negative weights across the mixed phase space.

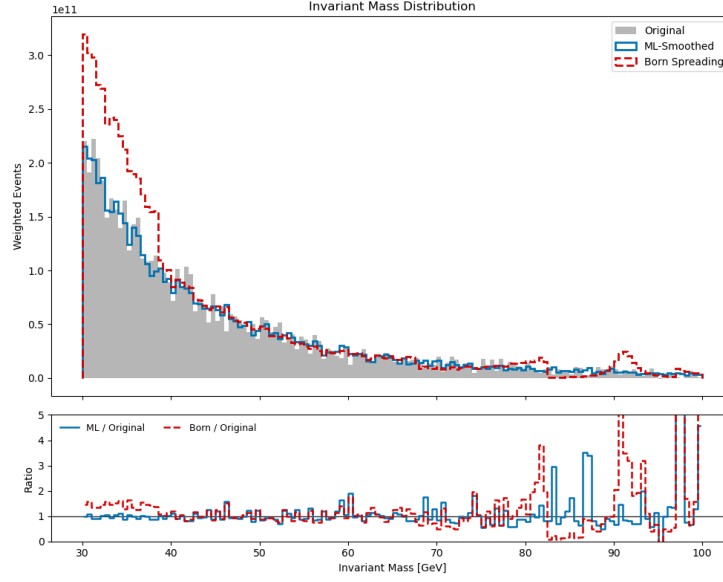


Figure 43: Invariant mass (m_{inv}) distribution of the mixed dataset. The modular BDT smoothly captures both the low-mass QCD spectrum and the sharp electroweak resonance.

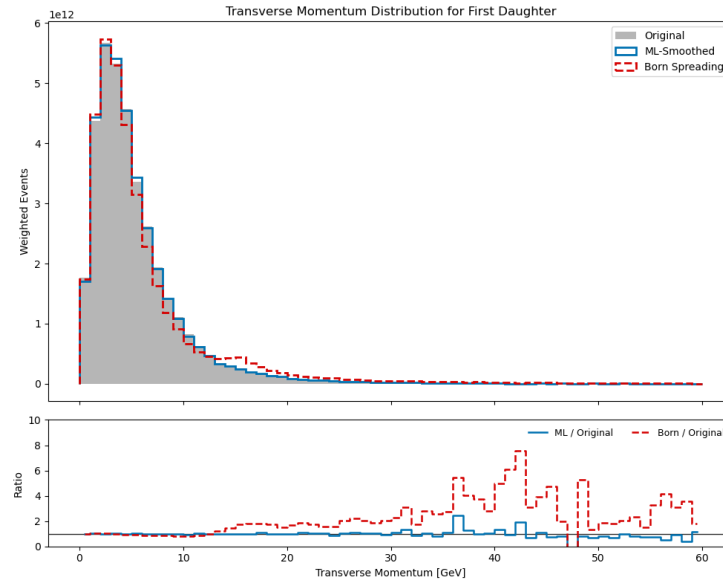


Figure 44: Transverse momentum (p_T) distribution of the combined Drell-Yan and $b\bar{b}$ events, showing perfect alignment with the NLO baseline.

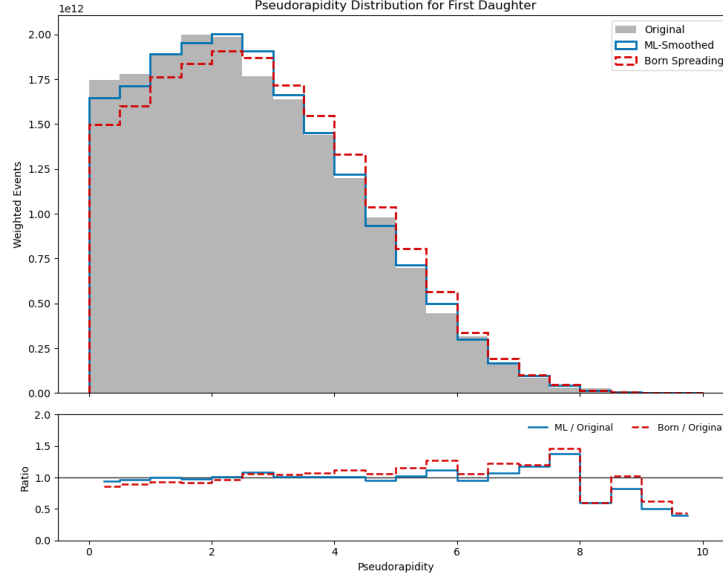


Figure 45: Pseudorapidity (η) spatial distribution confirming that the mixed geometric properties are perfectly conserved by the routed models.

4.4 Computational Efficiency and Effective Sample Size (ESS)

Beyond simply generating correct kinematic shapes, the ultimate value of any weight-smoothing framework is measured by its computational cost and statistical power. In Monte Carlo event generation, this performance is evaluated using two primary metrics: the processing time required to evaluate the events (CPU cost), and the Effective Sample Size (ESS), which measures the true statistical power of the final dataset.

Maximizing Statistical Power (ESS)

The most significant achievement of the Boosted Decision Tree (BDT) framework is the stabilization of the event weights. With the singular exception of the monolithic training attempt, the dedicated single-process models and the modular routing framework successfully reduced the negative weight fraction to exactly zero.

Because standard Next-to-Leading-Order (NLO) generation relies on severe negative subtractions, it inherently suffers from massive statistical penalties (e.g., a sample with a large negative fraction requires generating millions of extra events just to reach a baseline ESS). By completely eliminating the negative weight band, the BDT method pushes the ESS to its mathematical maximum.

Every single event generated and stored by the pipeline is physically useful.

Computational Inference Overhead

However, this massive statistical improvement comes with an additional computational cost. Evaluating a multi-dimensional decision-tree ensemble at every phase-space point is naturally more CPU-demanding than evaluating a straight-forward analytical expression.

During the $b\bar{b}$ production testing, the computational times were measured directly. The total CPU inference time for the XGBoost model evaluated to 0.1402 seconds, whereas the mathematical smearing of the traditional Born Spreading method required only 0.0177 seconds. This demonstrates that the machine learning inference step is roughly an order of magnitude slower than standard algebraic spreading.

The Global Efficiency Trade-Off

While the localized ML inference step is undeniably slower, this computational overhead is heavily outweighed by the global efficiency gains in the Monte Carlo generator.

In standard generation, the presence of sharp, high-variance weight spikes raises the maximum integration ceiling (h). This forces the acceptance-rejection algorithm to reject the vast majority of sampled points, wasting massive amounts of CPU time computing Matrix Elements that are ultimately discarded. Because the BDT model flattens and compresses the weight distribution so effectively, the generator can accept almost every point it samples. The dramatic reduction in wasted integration cycles, combined with the maximized ESS, makes the machine learning framework highly efficient overall, easily justifying the microsecond-level overhead of the tree inference.

5 Conclusion and Future Outlook

Conclusion

In this study, we introduced and tested a Boosted Decision Tree (BDT) framework designed to solve the negative-weight problem in Next-to-Leading-Order (NLO) Monte Carlo event generation. By treating the phase-space subtraction counter-terms as a continuous regression task, the machine learning approach effectively learned and smoothed the unphysical weight variations that naturally arise in NLO calculations.

The results demonstrate that this ML-smoothed framework is strictly superior to both standard Monte Carlo production and traditional Born Spreading. Standard generation suffers from severe statistical penalties and massive computational rejection rates due to localized weight spikes. Furthermore, while traditional Born Spreading can partially mitigate negative weight fractions, this study demonstrated that it explicitly fails to maintain the correct kinematic variables in combined, multi-process environments due to artificial over-smearing of the phase space boundaries.

In contrast, the BDT framework completely eliminated negative weights in both isolated and modular multi-process environments. Crucially, it achieved this while perfectly preserving the complex kinematic geometries of Drell-Yan, $b\bar{b}$, and $t\bar{t}$ interactions, maximizing the Effective Sample Size (ESS) and drastically improving the global efficiency of the event generator without compromising physical accuracy.

Future Outlook

While the modular BDT architecture provides a highly optimized solution for phase-space smoothing, there are several main areas for future development. First, the current pipeline can be readily expanded to include more Standard Model processes, including complex multi-jet and multi-boson final states.

Second, further research will focus on overcoming the massive differences in cross-section scale that limit the monolithic training architecture. Preliminary tests using advanced phase-space normalization have shown promising results in balancing mixed QCD and electroweak datasets. While these global scaling methods are still under active development, they offer a potential path toward a fully anonymous, single-model smoothing framework.

Finally, the computational efficiency of the pipeline can be further improved. Although the BDT inference step introduces a minor CPU overhead compared to simple spreading, this evaluation time can be significantly reduced. Parallelizing decision tree inference through multi-threading or GPU acceleration can further improve the performance of the machine learning pipeline for future high-throughput collider simulations.

References

- [1] Frixione, S., & Webber, B. R. (2002). Matching NLO QCD Computations and Parton Shower Simulations. *Journal of High Energy Physics*, 0206 (2002) 029.
- [2] Akhouri, R., et al. (1997). The KLN Theorem and Soft Radiation in Gauge Theories: Abelian Case. *arXiv preprint hep-ph/9702270v1*.
- [3] Che, Y. (2024). *Reducing Negative Weights in MC@NLO by Improved Implementation of Born Spreading*. (Bachelor thesis, Lund University).
- [4] Cairo, V. [ATLAS Collaboration] (2019). Probing $g \rightarrow b\bar{b}$ with inclusive jets and V+jets with ATLAS. *Proceedings of Science, PoS(LHCP2019)210*.
- [5] Frixione, S. (2005). MC@NLO: a tutorial. *Physics at TeV Colliders, Les Houches*.
- [6] Thijsen, J. *Computational Physics*. Cambridge University Press.
- [7] Cowan, G. (1998). *Statistical Data Analysis*. Oxford University Press.
- [8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>
- [9] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer.